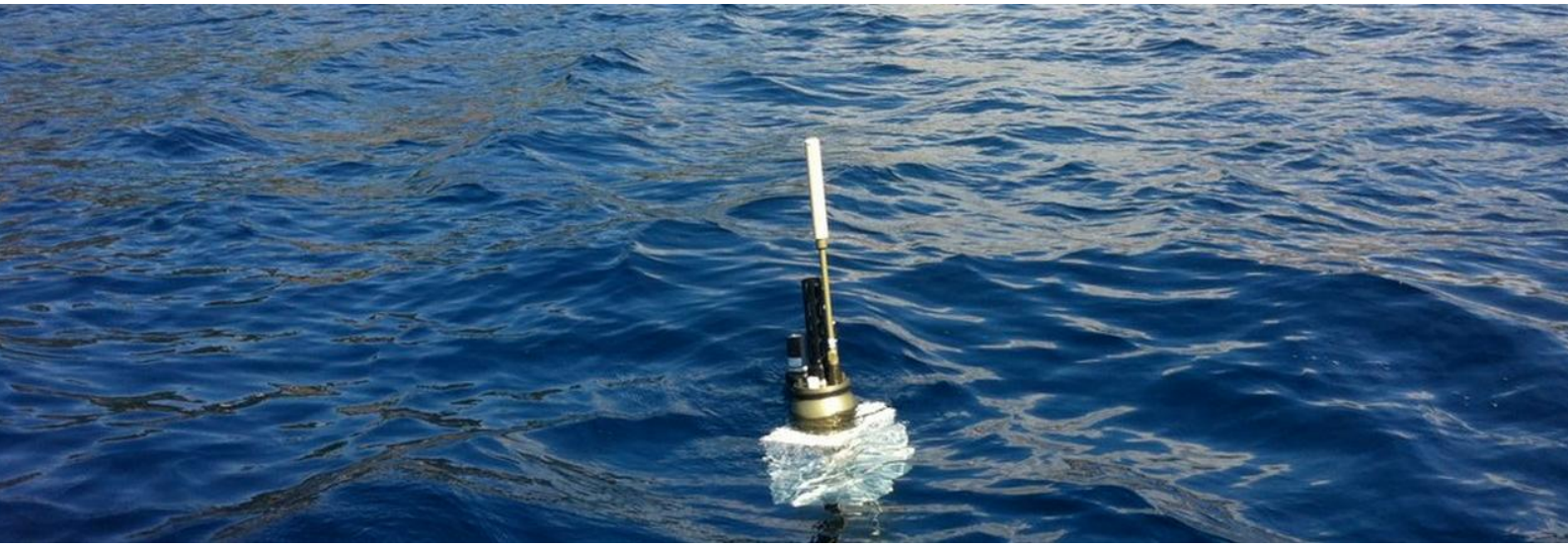
APMT Profiler - File management

AUTOMATED **M**ULTI-**T**ASK **P**ROFILER



33-16-046_APMT_File_Management
Revision 1.16 (2025-01-08)

Table of contents

Table of contents 2
1. Revision history 6
1.1 Firmware upgrade 9
2. File system 10
2.1 Memory organisation 10
2.2 File types 10
2.3 Main memory – Working memory 10
2.4 Extended memory – Storage memory (optional) 10
3. Configuration files 12
3.1 Local configuration via Bluetooth 12
3.2 Remote configuration via remote commands 12
3.3 Transmission of the configuration file 12
4. Technical files 13
4.1 Transmission of technical information 13
4.1.1. Self-test files 13
4.1.2. Mission files 13
4.1.3. End-of-life files 13
4.2 Technical information available 14
[SYSTEM] 14
[GPS] 14
[USER] 14
[ACTIVATION] 14
[PROFILE] 14
[DATA] 17
[POWER] 17
[ALARM] 18
[SENSOR_SUNA] 20
[SENSOR_SBEPH] 21
[SENSOR_UVP6] 21
[SENSOR_FLTRIDER] 21
5. Metadata files 23
5.1. Transmission of metadata information 23

5.2. Metadata information available 23
[PROFILER] 23
[TELECOM] 23
[BEHAVIOUR] 23
[HARDWARE] 23
[SENSOR_SBE41] 24
[SENSOR_DO] 24
[SENSOR_OCR] 24
[SENSOR_ECO] 25
[SENSOR_ECOv2] 25
[SENSOR_CROVER] 25
[SENSOR_SBEPH] 25
[SENSOR_SUNA] 25
[SENSOR_UVP6] 26
[SENSOR_RAMSES] 26
[SENSOR_RAMSES2] 26
[SENSOR_OPUS] 26
[SENSOR_MPE] 27
[SENSOR_HYDROC] 27
[SENSOR_IMU] 27
[SENSOR_FLTRIDER] 27
[SENSOR_PAL] 28
[SENSOR_TRIDENTE] 28
[SENSOR_SENSO] 28
[SENSOR_RINKO] 29
6. Sensor data files 31
6.1. Transmission of sensor data 31
6.2. ID tags 31
6.2.1. Navigation phase identification 31
6.2.2. Processing type identification 32
6.3. Recording in text format 32
6.4. Recording in binary format 33
6.4.1. Encoding format 33
6.4.2. File structure 33

6.4.3. Timestamping 35
6.5. SBE41 sensor 36
6.5.1. Data encoding 36
6.5.2. Standard binary format 36
6.5.3. Extended binary format 41
6.6. DO sensor 47
6.7. OCR sensor 52
6.8. ECO sensor 57
6.9. ECOv2 sensor 62
6.10. CROVER sensor 67
6.11. SBEPH sensor 70
6.12. SUNA sensor 74
6.13. UVP6 sensor (LPMv2) 78
6.14. UVP6 sensor (TAXOv2) 81
6.15. UVP6 sensor (BLACKv2) 85
6.16. EXTTRIG sensor 87
6.17. RAMSESv2/RAMSES2v2 sensor 88
6.18. OPUS sensor (LIGHTv2) 91
6.19. OPUS sensor (BLACKv2) 94
6.20. MPE sensor 96
6.21. HYDROC sensor (M or C) 98
6.22. IMU sensor (R) 101
6.23. IMU sensor (TH) 104
6.24. WAVE sensor (R) 106
6.25. FLTRIDER sensor 108
6.26. PAL sensor (R+W) 109
6.27. PAL sensor (HR) 110
6.28. PAL sensor (HD) 111
6.29. PAL sensor (F) 112
6.30. TRIDENTE sensor (3 channels) 114
6.31. TRIDENTE sensor (9 channels) 116
6.32. SENSO sensor (2 channels) 118
6.33. SENSO sensor (3 channels) 120
6.34. RINKO sensor 122

7. Annexes 124
7.1. Annex A – Argo metadata table 124

1. Revision history

Revision	Release date	Notes	Author
1.0	2017-09-04	Original	C. SCHAEFFER
1.1	2018-02-15	Upgrade of technical information: - Adding internal temperature information in [SYSTEM] - Adding pressure activation information in [ACTIVATION] - Adding nke ID information in [USER] - Changing grounding information during navigation phases in [PROFILE] and [ALARM] - Changing feedback information in [ALARM]	C. SCHAEFFER
1.2	2018-11-19	Upgrade of technical information: - Adding pressure reference in “VBatt peak min” tag - Adding USEA identification in [USER] - Adding sensor identification [SENSOR_xxx] - Adding surface sample count in [DATA] - Adding sensor tag in [POWER] - Adding alarms flag in [ALARM] Upgrade of sensor data files: - Adding navigation phase [SURFACE] - Adding decimated raw treatment tag (DW) - Adding a 100 dbar offset in SBE41 pressure - Adding DO, OCR, ECO, C-ROVER, SBEPH, SUNA data format	C. SCHAEFFER
1.3	2019-12-12	Upgrade of technical information: - Adding metadata files - Removing redundant information [USER] - Removing redundant information [SENSOR_xxx] - Changing “kB” into “KB” Upgrade of sensor data files - Adding UVP6 data formats Update SBE41 *.hex standard format: - Adding time stamping for each record Upgrade of technical information: - Adding ice capabilities alarms	C. SCHAEFFER
1.4	2020-01-20	Upgrade of sensor data files - Adding EXTTRIG data format	C. SCHAEFFER
1.5	2020-02-11	Upgrade of sensor data files - Adding ECOv2 data format - Adding technical and metadata information related to ECOv2	C. SCHAEFFER
1.6	2020-03-12	Upgrade of technical information - Adding ice braking and perigee in [PROFILE] - Adding alarm flag in [ALARM]	C. SCHAEFFER
1.7	2020-08-26	The following are the changes to this manual from the previous revision to document the firmware update: **APMT v1.08.004 - USEA v1.01.004** Upgrade of technical and sensor data files	C. SCHAEFFER

		- Adding RAMSES data format	
1.8	2020-09-30	Upgrade of metadata file: - Changing profiler model format - Adding APMT's SD card information - Adding BATTERY information - Adding SBE41 firmware information	C. SCHAEFFER
1.9	2020-12-16	The following are the changes to this manual from the previous revision to document the firmware update: **APMT v1.08.005 - USEA v1.01.005** Upgrade of technical and sensor data files - Adding OPUS data format Upgrade of metadata file: - Adding PROFILER.ID information - Adding TELECOM.Login information - Adding ECO.Type information - Adding EXTENSION_BOARD information - Adding HULL information - Adding SBE41 calibration coefficients - Adding CONTROL_BOARD and MEASURE_BOARD SN information	C. SCHAEFFER
1.10	2021-04-07	The following are the changes to this manual from the previous revision to document the firmware update: **APMT v1.09.001 - USEA v1.02.001** Upgrade of technical and sensor data files - Adding MPE data format - Adding [POWER].Measure information	C. SCHAEFFER
1.11	2021-09-02	The following are the changes to this manual from the previous revision to document the firmware update: **APMT v1.09.002 - USEA v1.02.002** Upgrade of technical and sensor data files - Adding SBEPH technical information - Changing UVP6 LPM, TXO and BLK data format for UVP6 version 2 - Adding UVP6-TXO technical information - Deleting UVP6-TX1 and UVP6-TX2 - Changing UVP6 metadata format	C. SCHAEFFER
1.12	2021-11-09	The following are the changes to this manual from the previous revision to document the firmware update: **APMT v1.09.003 - USEA v1.02.003** Upgrade of technical and sensor data files - Adding HYDROC technical information - Adding HYDROC data format	C. SCHAEFFER
1.13	2022-02-07	The following are the changes to this manual from the previous revision to document the firmware update: **APMT v1.09.004 - USEA v1.02.004** Upgrade of technical and sensor data files	C. SCHAEFFER

		- Adding RAMSES2 technical information - Adding RAMSES2 data format - Adding IMU technical information - Adding IMU data format - Changing UVP6-TXO data format - Changing HYDROC's pump power resolution	
1.14	2022-06-16	Upgrade of technical and sensor data files - Updating IMU data format - Adding WAVE technical information - Adding WAVE format - Adding decoding example for UVP6-TXO data format	C. SCHAEFFER
1.15	2023-01-04	The following are the changes to this manual from the previous revision to document the firmware update: **APMT v1.09.005 - USEA v1.02.005** Upgrade of technical and sensor data files - Updating ISA alarm information - Adding transmission quality factor - Adding FLTRIDER technical information - Adding FLTRIDER data format - Adding IMEI and BEHAVIOUR metadata information - Adding appendix A (Argo export table)	C. SCHAEFFER
1.16	2025-01-08	The following are the changes to this manual from the previous revision to document the firmware update: **APMT v1.09.008 - USEA v1.02.008** Upgrade of technical and sensor data files - Updating DATA information - Adding PAL, TRIDENTE, SENSO and RINKO technical information - Adding PAL, TRIDENTE, SENSO and RINKO data format - Adding Event-Driven behaviour in technical information	C. SCHAEFFER

1.1 Firmware upgrade

The following are the main changes to embedded software:

APMT	USEA	Notes
1.08.002	1.01.002	Adding Under Ice capabilities
1.08.003	1.01.003	Adding ECOv2 sensor
1.08.004	1.01.004	Adding RAMSES sensor
1.08.005	1.01.005	Adding OPUS sensor
1.09.000	1.02.000	Adding Multi-Parking capabilities
1.09.001	1.02.001	Adding MPE sensor
1.09.002	1.02.002	Updating UVP6 sensor
1.09.003	1.02.003	Adding HYDROC sensor
1.09.004	1.02.004	Adding RAMSES2 sensor Adding IMU sensor
1.09.005	1.02.005	Updating IMU sensor Adding FLTRIDER sensor
1.09.006	1.02.006	Adding PAL sensor Adding Event-Driven capabilities
1.09.007	1.02.007	Adding USEA-Payload capabilities
1.09.008	1.02.008	Adding TRIDENTE sensor Adding SENSO sensor Adding RINKO sensor

2. File system

Float management is based on a specific file system developed by nke. This file system allows information and data to be stored when the float is turned off, together with the mission configuration.

These files can be accessed by an FTP client via the Bluetooth connection.

2.1 Memory organisation

The memory space is divided into different areas intended for different uses. In particular, the following spaces can be found:

- Reserved: restricted system use to store file manipulation and management information
- Working: used for temporary storage (awaiting transmission).
- Storage (optional): used to expand the temporary storage capacity of the workspace

2.2 File types

The system uses various file types, as needed:

- Configuration file for mission and sensor settings
- Sensor data files (raw, processed and transmitted data)
- Technical data files
- Metadata files (for sensor specific identification)
- Execution trace files (mission progress monitoring)
- Specific files (restricted system use)

2.3 Main memory – Working memory

The working memory (fast access) is itself divided into two physical spaces:

- Area for repeated access files: used for files with recurring access (configuration file...)
- Area for temporary files: used mainly for the generation of sensor information and data during the various navigation phases (a process that automatically deletes the oldest files in favour of new ones allows a minimum free memory space to be maintained)

2.4 Extended memory – Storage memory (optional)

When the application requires it, the use of an additional memory (slow access) allows the storage capacity of the system to be increased.

In this case, the files (awaiting transmission) deleted from the working memory are moved to the extended memory. They will then be repatriated to the working memory (for transmission) when the main memory space is sufficient.

3. Configuration files

The mission configuration is saved as a text file organised by sections and tags.

3.1 Local configuration via Bluetooth

During the mission configuration phase performed locally via Bluetooth, this file can be written and/or modified by the user. It can be manually uploaded in full using an FTP client or partially modified with a TELNET command interpreter.

3.2 Remote configuration via remote commands

When the mission is in progress, the file can be modified by sending remote commands via the satellite communication system (RUDICS...).

3.3 Transmission of the configuration file

The system transmits the current configuration file at various stages of the mission:

- At the start of the mission (initial configuration)
- Upon a change of configuration following the application of a remote command or pre-programmed change (script)

The various files generated are named using the format "xxxx_ccc_mm_apmt.ini", with:

- "xxxx": float's hexadecimal serial number (4 characters)
- "ccc": cycle number (3 characters)
- "mm": pattern number (2 characters)

4. Technical files

The various pieces of technical information associated with a profile are saved in a text file organised by sections.

4.1 Transmission of technical information

The system transmits technical files at various stages of the mission:

- During system verification prior to the mission (self-test)
- During the mission at the end of each pattern
- At end of life

4.1.1. Self-test files

The files generated are named using the format "xxxx_ccc_autotest_nnnnn.txt", with:

- "xxxx": float's hexadecimal serial number (4 characters)
- "ccc": cycle number (3 characters)
- "nnnnn": identification of self-test file number associated with the cycle (5 characters)

4.1.2. Mission files

The files generated are named using the format "xxxx_ccc_mm_technical.txt":

- "xxxx": float's hexadecimal serial number (4 characters)
- "ccc": cycle number (3 characters)
- "mm": pattern number (2 characters)

Note: A pattern numbered "00" corresponds to a pre-mission file.

4.1.3. End-of-life files

The various files generated are named using the format "xxxx_ccc_mm_default_nnnnn.txt", with:

- "xxxx": float's hexadecimal serial number (4 characters)
- "ccc": cycle number (3 characters)
- "mm": pattern number (2 characters)
- "nnnnn": identification of end-of-life file number associated with the cycle/pattern pair (5 characters)

4.2 Technical information available

The data is organised in sections. The various sections and fields that compose them are optional and may not be present in the files.

[SYSTEM]

Information	Format
External pressure offset	Pe offset=xx.xx dbar
Internal pressure	Pi=xxx.x mbar
No-load battery voltage	Vbatt=xx.x V
Battery voltage under load (min.)	Vbatt peak min=xx.x V (xx dbar)
External temperature (air)	Te air=xx.x degC
External pressure (air)	Pe air=xx.x dbar
Internal temperature	Ti=xx.x degC

[GPS]

Information	Format
Time stamping, position (*) and clock drift (**)	UTC=yy-mm-dd hh:mm:ss Lat=ddmm.mmmmh Long=dddmm.mmmmh Clock drift=xx.xxx s

(*) Geographic coordinates are coded in DM format (degrees, minutes and decimal minutes – NMEA standard). For example: “Lat=4747.846N Long=00317.160W” represents 47°47.846’N 3°17.160’W coordinates.

(**) Drift = Float time - GPS time. The offset is reset to 0 after every GPS retiming.

[USER]

Information	Format
nke ID	WC=xxxxxxxxxx

[ACTIVATION]

Information	Format
Pressure activation start	UTC=yy-mm-dd hh:mm:ss Detection start
Pressure activation stop	UTC=yy-mm-dd hh:mm:ss Detection stop

[PROFILE]

Information	Format
Emergence reduction: - Start date/time - Oil volume transferred - Nb. solenoid valve actions	UTC=yy-mm-dd hh:mm:ss Flotation=xxx.x cm3 (xx)
First stabilization - Date/time - Depth	UTC=yy-mm-dd hh:mm:ss First stabilization=xxx dbar
Park descent: - Start date/time - Oil volume transferred - Nb. solenoid valve actions	UTC=yy-mm-dd hh:mm:ss Descent=xxx.x cm3 (xx) UTC=yy-mm-dd hh:mm:ss Descent_y=xxx.x cm3 (xx) (****)
Grounding park descent:	UTC=yy-mm-dd hh:mm:ss Grounding Descent=xxx dbar



- Start date/time - Grounding pressure	UTC=yy-mm-dd hh:mm:ss Grounding Descent_y=xxx dbar (****)
Grounding park descent: - End date/time - Oil volume transferred	UTC=yy-mm-dd hh:mm:ss Grounding Descent escape=xxx.x cm3
Park drift: - Start date/time - Min. depth - Max. depth - Nb. solenoid valve actions - Nb. pump actions - Nb. set point inputs - Nb. set point outputs	UTC=yy-mm-dd hh:mm:ss Park=xxx/xxx dbar (xx/xx) stability=x/x UTC=yy-mm-dd hh:mm:ss Park_y=xxx/xxx dbar (xx/xx) stability=x/x (****)
Stabilized park drift: - Stabilization date/time - Stabilization depth	UTC=yy-mm-dd hh:mm:ss Park stabilization=xxx dbar UTC=yy-mm-dd hh:mm:ss Park_y stabilization=xxx dbar (****)
Grounding park drift: - Start date/time - Grounding number (1-5) - Grounding pressure	UTC=yy-mm-dd hh:mm:ss Grounding Park x=xxx dbar
Grounding park drift: - End date/time - Grounding number (1-5) - Oil volume transferred	UTC=yy-mm-dd hh:mm:ss Grounding Park x escape=xxx.x cm3
Measurement descent: - Start date/time - Oil volume transferred - Nb. solenoid valve actions	UTC=yy-mm-dd hh:mm:ss Deep profile=xxx.x cm3 (xx)
Grounding measurement descent: - Start date/time - Grounding pressure	UTC=yy-mm-dd hh:mm:ss Grounding Deep profile=xxx dbar
Grounding measurement descent: - End date/time - Oil volume transferred	UTC=yy-mm-dd hh:mm:ss Grounding Deep profile escape=xxx.x cm3
Measurement drift: - Start date/time - Min. depth - Max. depth - Nb. solenoid valve actions - Nb. pump actions - Nb. set point inputs - Nb. set point outputs	UTC=yy-mm-dd hh:mm:ss Short Park=xxx/xxx dbar (xx/xx) stability=x/x
Grounding short park drift: - Start date/time - Grounding number (1-5) - Grounding pressure	UTC=yy-mm-dd hh:mm:ss Grounding Short park x=xxx dbar
Grounding short park drift: - End date/time - Grounding number (1-5) - Oil volume transferred	UTC=yy-mm-dd hh:mm:ss Grounding Short park x escape=xxx.x cm3
Ascent (standard):	UTC=yy-mm-dd hh:mm:ss Ascent=xxx.x cm3 (xx/xx) from xxx



- Start date/time - Oil volume transferred - Nb. pump actions (total) - Nb. actions for take-off - Maximum depth	dbar
Ascent (slow): - Start date/time - Oil volume transferred - Nb. pump actions	UTC=yy-mm-dd hh:mm:ss Ascent (slowly)=xxx.x cm3 (xx)
Ascent (resume): - Start date/time - Oil volume transferred - Nb. pump actions	UTC=yy-mm-dd hh:mm:ss Ascent (resume)=xxx.x cm3 (xx)
Ascent (end) - End date/time	UTC=yy-mm-dd hh:mm:ss Ascent end
Ice abort: - Abort date/time - Oil volume transferred - Abort pressure	UTC=yy-mm-dd hh:mm:ss Abort=xxx.x cm3 at xxx dbar
Ice perigee: - Perigee date/time - Perigee pressure	UTC=yy-mm-dd hh:mm:ss Perigee=xxx dbar
Surface - Start date/time	UTC=yy-mm-dd hh:mm:ss Surface
Hanging: - Start date/time - Hanging pressure	UTC=yy-mm-dd hh:mm:ss Hanging=xxx dbar
Hanging: - End date/time	UTC=yy-mm-dd hh:mm:ss Hanging escape
Park drift (ice): - Start date/time - Min. depth - Max. depth - Nb. solenoid valve actions - Nb. pump actions - Nb. set point inputs - Nb. set point outputs	UTC=yy-mm-dd hh:mm:ss Ice Park=xxx/xxx dbar (xx/xx) stability=x/x
Stabilized park drift (ice): - Stabilization date/time - Stabilization depth	UTC=yy-mm-dd hh:mm:ss Ice Park stabilization=xxx dbar
Pressure switch activation	UTC=yy-mm-dd hh:mm:ss Pressure switch activation
Emergency ascent - Start date/time	UTC=yy-mm-dd hh:mm:ss Emergency ascent

Typical rate of descent can be calculated with: “Park drift” UTC - “Emergence reduction” UTC
Typical rate of ascent can be calculated with: “Ascent (end)” UTC – “Ascent (standard)” UTC

[DATA]

Information	Format
Data transmission (*): - Total data size - Nb. files - Average baud rate - Nb. sessions	Upload=xx.x KB of x file(s) at x.x KB/min in x session(s)
Remote command reception (*): - Nb. accepted - Nb. refused - Nb. unknown	Download=command file (x accepted, x refused, x unknown)
Payload configuration download (*)	Download=payload file
Script download (*)	Download=script file
Event download (*)	Download=event file
Payload download (*)	Download=payload file
Number of files associated with the pattern (**)	Pattern=x files
Number of points per sensor: - Sensor name [SBE41, DO, OCR, ECO, ECOv2, CROVER, SBEPH, SUNA, UVP6-LPM, UVP6-TXO, UVP6-BLK, EXT-TRIG, RAMSES, RAMSES2, OPUS-LGT, OPUS-BLK,MPE, HYDROC-M, HYDROC-C, IMU, WAVE, PAL, TRIDENTE, SENSO, RINKO] - Nb. park descent - Nb. park drift - Nb. measurement descent - Nb. measurement drift - Nb. ascent - Nb. surface - Nb. subsurface (***) - Nb. decimated - Nb. total	xxxxx=x/x/x/x/x/x points (x/x) or xxxxx=x/x/x/x/x/x/x points (x/x) (***)
Transmission quality survey (*)	SQ=xxx sec QC=x.x/x.x/x.x

(*) Information from the previous transmission session.
(**) The following files are not taken into account: technical for self-test or pre-mission, configuration nor command/script acknowledgement. The number of files is expressed prior to the files being broken down for transmission.
(***) SBE41 only
(****) With multi-parking navigation scheme

[POWER]

Information	Format
Pattern duration	Pattern=x min



Processing/standby ratio	Treatment=xx % Measure=xx %
Cumulated hydraulic activations: - Solenoid valve - Pump	EV/Pump=xxx/xxx cs
Cumulated modem activations (*)	Transmission=xxx min
Cumulated GPS activations	GPS=xxx s
Cumulated sensor activations: - Sensor name [SBE41, DO, OCR, ECO, ECOv2, CROVER, SBEPH, SUNA, UVP6, EXT-TRIG, RAMSES, RAMSES2, OPUS, MPE, HYDROC, IMU, FLTRIDER, PAL, TRIDENTE, SENSO, RINKO] - Duration	xxxx=xxx min

(*) Information from the previous transmission session.

[ALARM]

Information	Format
Deployment	
Start up	Power-on
Invalid configuration	Bad configuration
Excessively heavy float	Flotation (heavy)
Excessively light float	Flotation (light)
Self-test failure: - Source(s) xxx among "FRAM, FLASH, RTC, Vbatt, Pi, Pe, SBE41-Cutoff, SBE41-Offset, GPS, Payload, USEA, Sensor(xxx), Transmitter"	Autotest fail=xxx, …
State	
No-load battery voltage low	Vbatt low
Battery voltage under load low	Vbatt peak low
Low external pressure	Pe low (xxx dbar)
High external pressure	Pe high (xxx dbar)
External pressure fault	Pe default
External breaking pressure	Pe broken
Gear skip	Pe SR high
High internal pressure	Pi high
Presence of water	Water inside
Navigation	
Grounding during park descent	Grounding Descent (xxx dbar) Grounding Descent_y (xxx dbar) (****)
Grounding during park drift	Grounding Park x (xxx dbar)
Grounding during measurement	Grounding Deep profile (xxx dbar)



descent	
Grounding during short park drift	Grounding Short park x (xxx dbar)
Hanging during ascent	Hanging (xxx dbar)
Braking during descent	Braking
Operating errors	
System fault	System
Payload board fault	Payload
GPS fault	GPS
Hydraulic fault	Hydraulic
ADC fault	ADC
File fault	File (skip)
RTC fault	RTC
Pressure switch fault	Pressure switch
Ice capabilities	
Ice avoidance (ISA detection): - Median temperature	Ice (ISA), Tm=-x.xxx degC
Ice avoidance (collision detection)	Ice (collision)
Ice avoidance (aborting profile)	Ice (abort)
Ice avoidance (cover detection) (*)	Ice (cover)
Ice avoidance (no surface period)	Ice (period)
Event-Driven capabilities	
Event (aborting profile)	Event (abort)
Event (restoring mission)	Event (mission)
Event detection (rain)	Event (rain)
Event detection (wind)	Event (wind)
Event detection (backscatter)	Event (backscatter)
Mode switch	
End of life: - Source xxx among "Pi high, Pe broken, Pe high, Vbatt low, Vbatt peak low, Water inside, Flotation (heavy), Flotation (light)"	End of life (xxx)
Rescue procedure	Rescue
Feedback: - Type xxx among "Early profile, Early surface, Abort profile, Abort cycle 0, Abort cycle 1, Go to deep, Standard speed, Lower speed" - Nb. accepted - Nb. refused	Feedback=xxx (x accepted, x refused) Feedback=xxx (x accepted, x refused) …
Sensors	
Bad value in data frame: - Sensor name [SBE41, DO, OCR, ECO, ECOv2, CROVER, SBEPH, SUNA, UVP6, RAMSES, RAMSES2,	xxx value (xxx)

OPUS, MPE, HYDROC, IMU, PAL, TRIDENTE, SENSO, RINKO] - Error count	
No reply/framing error: - Sensor name [SBE41, DO, OCR, ECO, ECOv2, CROVER, SBEPH, SUNA, UVP6, RAMSES, RAMSES2, OPUS, MPE, HYDROC, IMU, PAL, TRIDENTE, SENSO, RINKO] - Error count	xxx default (xxx)
Repetitive default: - Sensor name [SBE41, DO, OCR, ECO, ECOv2, CROVER, SBEPH, SUNA, UVP6, RAMSES, RAMSES2, OPUS, MPE, HYDROC, IMU, FLTRIDER, PAL, TRIDENTE, SENSO, RINKO]	xxx broken
Sequence error: - Sensor name [HYDROC, FLTRIDER] - Error code	xxx sequence (0xYY/0xYYYY)
Data size: - Sensor name [SBE41, DO, OCR, ECO, ECOv2, CROVER, SBEPH, SUNA, UVP6-LPM, UVP6-TXO, UVP6-BLK, EXT-TRIG, RAMSES, RAMSES2, OPUS-LGT, OPUS-BLK, MPE,HYDROC-M, HYDROC-C, IMU, FLTRIDER, PAL, TRIDENTE, SENSO, RINKO] - Data size	xxx size (xxx.x KB)

(*) Information from the previous profile.

[SENSOR_SUNA]

Information	Format
Counters: - Sample counter - Power cycle counter - Error counter	Counters=892340/852111/16
Power supply: - Voltage - Intensity	Power supply=9.53 V/0.663 A

[SENSOR_SBEPH]

Information	Format
First ascent frame: - Vref - Vk - Ik - Ib	First=x.xxxxxx/x.xxxxxx/x.xxxxxx/x.xxxxxx at xxxx.x dbar
Last ascent frame: - Vref - Vk - Ik - Ib	Last=x.xxxxxx/x.xxxxxx/x.xxxxxx/x.xxxxxx at x.x dbar

[SENSOR_UVP6]

Information	Format
SD memory free space	Available space=xxx.x GB

[SENSOR_FLTRIDER]

Information	Format
SD memory free space	Available space=xxx.x GB

Example:

[USER]
WC=85E386B7C7
[SYSTEM]
Pe offset=0.00 dbar
Pe air=0.00 dbar
Pi=750.3 mbar
Vbatt=10.7 V
Vbatt peak min=10.0 V (0 dbar)
[GPS]
UTC=18-11-20 12:17:24 Lat=4310.35000S Long=00555.02500W Clock drift=+0.000 s
[PROFILE]
UTC=18-11-20 10:12:22 Flotation=10.7 cm3 (1)
UTC=18-11-20 10:34:03 First stabilization=9 dbar
UTC=18-11-20 10:23:45 Descent=12.2 cm3 (4)
UTC=18-11-20 11:33:06 Park=68/68 dbar (0/0) stability=0/0
UTC=18-11-20 11:33:08 Deep profile=0.0 cm3 (0)
UTC=18-11-20 11:34:06 Short Park=68/68 dbar (0/0) stability=0/0
UTC=18-11-20 11:34:07 Ascent=24.6 cm3 (12/4) from 69 dbar
UTC=18-11-20 12:00:25 Ascent end
UTC=18-11-20 12:10:25 Surface
[DATA]
Upload=4.8 KB of 3 file(s) at 10.2 KB/min in 1 session(s)
Pattern=6 files
SBE41=47/0/0/0/30/6/1 points
OCR=95/0/0/0/33/0 points
ECO=86/0/0/0/30/0 points
SBEPH=58/0/0/0/32/0 points
SUNA=7/0/0/0/4/0 points
[POWER]
Pattern=126 min
Treatment=1 %
EV/Pump=337/382 cs
SBE41=115 min
Transmission=2 min
GPS=260 s
OCR=6 min
ECO=6 min
SBEPH=6 min
SUNA=1 min
[SENSOR_SUNA]
Counters=892340/852111/16
Power supply=9.53 V/0.663 A

5. Metadata files

5.1.Transmission of metadata information

Metadata files are transmitted once at the beginning of the mission.

The files generated are named using the format "xxxx_ccc_mm_metadata.xml", with:

- "xxxx": float's hexadecimal serial number (4 characters)
- "ccc": cycle number (3 characters)
- "mm": pattern number (2 characters)

5.2.Metadata information available

Metadata information are organised in XML format.

[PROFILER]

Information	Format
Identification: - Serial number - Identifier - Model	<PROFILER SN=”P5XXXX-XXXXXXX” ID=”0xXXXX” Model=”PROVOR_V”/>

[TELECOM]

Information	Format
Identification: - Type - SIM card ID - Modem ID - Account login	<TELECOM Type=”IRIDIUM” CID=” xxxxxxxxxxxxxxx” IMEI=” xxxxxxxxxxxxxxx” Login=”xxxxx”/>

[BEHAVIOUR]

Information	Format
Behaviour information: - Options List - Energy estimation model	<BEHAVIOUR Options=”NONE” Energy=”Model B”/>

[HARDWARE]

Information	Format
Control board identification: - Model - Serial number - Firmware version - SD card status	<CONTROL_BOARD Model=”APMT” SN=”CYYMMDD-XXXX” Firmware=”x.xx.xxx” SDCard=”Installed/Not installed”/>
Measure board identification: - Model - Serial number - Firmware version	<MEASURE_BOARD Model=”USEA” SN=”CYYMMDD-XXXX” Firmware=”x.xx.xxx”/>

Extension board identification: - Model	<EXTENSION_BOARD Model=”EXTEND”/>
Battery configuration: - Pack number “x” information - Type - Voltage - capacity	<PACK_x Type=”Lithium” Voltage=”10.8 V” Capacity=”xxx Ah”/>
Hull configuration: - Model	<HULL Model=”Type x”/>

[SENSOR_SBE41]

Information	Format
Identification: - Serial number - Model	<SENSOR SN=”XXXXX” Model=”SBE41-CP”/>
Sensor board identification: - Firmware version	<SBE41_BOARD Firmware=”x.x.x”/>
Pressure sensor Identification: - Serial number	<SENSOR_PRESSURE SN="XXXXX”/>
Temperature correction: - Coefficients	<TEMPERATURE_COEFF ta0="xxxx" ta1="xxxx" ta2="xxxx" ta3="xxxx"/>
Conductivity correction: - Coefficients	<CONDUCTIVITY_COEFF g="xxxx" h="xxxx" i=”xxxx" j="xxxx" ctcor="xxxx" cpcor="xxxx" wbotc="xxxx"/>
Pressure correction: - Coefficients	<PRESSURE_COEFF pa0="xxxx" pa1="xxxx" pa2="xxxx" ptca0="xxxx" ptca1="xxxx" ptca2="xxxx" ptcb0="xxxx" ptcb1="xxxx" ptcb2="xxxx" ptha0="xxxx" ptha1="xxxx" ptha2="xxxx" poffset="xxxx"/>

[SENSOR_DO]

Information	Format
Identification: - Serial number - Model	<SENSOR SN=”XXXXX” Model=”DOxxx”/>
Phase correction: - Coefficient “c0”	<PHASE_COEFF c0="xxxx"/>
SVU foil correction: - Coefficient “c0” - ... - Coefficient “c6”	<SVU_FOIL_COEFF c0="xxxx" c1="xxxx" ... c6="xxxx"/>

[SENSOR_OCR]

Information	Format
Identification: - Serial number - Model	<SENSOR SN=”XXXXX” Model=”OCRxxx”/>
Channel correction: - Index [4,7 or 14] - Coefficient “a0” - Coefficient “a1”	<CHANNEL_xx a0=”x.xxxx” a1=”x.xxxx” im=”x.xxxx”/>

- Coefficient “im”	

[SENSOR_ECO]

Information	Format
Identification: - Serial number - Model - Type	<SENSOR SN=”XXXXX” Model=”ECOxxx” Type=”xxxxxx”/>
Channel correction: - Index [1-3] - Scale factor “sf” - Dark count “dc”	<CHANNEL_xx sf=”x.xxxx” dc=”xxxxxxx”/>

[SENSOR_ECOv2]

Information	Format
Identification: - Serial number - Model	<SENSOR SN=”XXXXX” Model=”ECOxxx”/>
Channel correction: - Index [1-3] - Scale factor “sf” - Dark count “dc”	<CHANNEL_xx sf=”x.xxxx” dc=”xxxxxxx”/>

[SENSOR_CROVER]

Information	Format
Identification: - Serial number	<SENSOR SN=”XXXXX”/>
Path length: - X value “pth” (cm)	<PATH_LENGTH pth="xx.xx"/>
Calibration: - CSCcal value “cln”	<CALIBRATION cln="xxxxx"/>

[SENSOR_SBEPH]

Information	Format
Identification: - Serial number	<SENSOR SN=”XXXXX”/>

[SENSOR_SUNA]

Information	Format
Identification: - Serial number - Model - Output pixel begin - Output pixel end	<SENSOR SN=”XXXXX” Model=”SUNA-V2” Spectrum=”Output pixels xx-xx”/>
Sensor board identification: - Firmware version	<SUNA_BOARD Firmware=”x.x.xx”/>
Spectrometer: - Integration time “spintper”	<SPECTROMETER spintper=”xxx”/>

[SENSOR_UVP6]

Information	Format
Identification: - Serial number - Model	<SENSOR SN=”XXXXXXXXXXX” Model=”UVP6-LP”/>
HW configuration	<HW_CONF frame=”xxxxxxxxxx”>
ACQ configuration: - Index [0-9]	<ACQ_CONF_x frame=”xxxxxxxxxx”>
TAXO configuration: - Index [0-9]	<TAXO_CONF_x frame=”xxxxxxxxxx”>

[SENSOR_RAMSES]

Information	Format
Identification: - Serial number	<SENSOR SN=”XXXXX”/>
Sensor board identification: - Firmware version	<RAMSES_BOARD Firmware=”x.x.xx”/>
Spectrum: - Wavelengths number - Wavelengths	<SPECTRUM Length=”xx” Wavelengths=”xx.xxx,[...],xx.xxx”/>

[SENSOR_RAMSES2]

Information	Format
Identification: - Serial number	<SENSOR SN=”XXXXX”/>
Sensor board identification: - Firmware version	<RAMSES_BOARD Firmware=”x.x.xx”/>
Spectrum: - Wavelengths number - Wavelengths	<SPECTRUM Length=”xx” Wavelengths=”xx.xxx,[...],xx.xxx”/>

[SENSOR_OPUS]

Information	Format
Identification: - Serial number	<SENSOR SN=”XXXXX”/>
Sensor board identification: - Firmware version	<OPUS_BOARD Firmware=”x.x.xx”/>
Spectrum: - Wavelengths number - Wavelengths	<SPECTRUM Length=”xxx” Wavelengths=”xx.xxx,[...],xx.xxx”/>
Lamp Identification: - Serial number	<SENSOR_LAMP SN="xxxxx”/>
Spectrometer: - Optical path size	<SPECTROMETER Path=”xx mm”/>
Waterbase reference spectrum: - Wavelengths number - Intensities	<WATERBASE Length=”xxx” Intensities=”xx.x,[...],xx.x”/>

[SENSOR_MPE]

Information	Format
Identification: - Serial number - Type	<SENSOR SN=”XXXXX” Type=”xxxx”/>
Acquisition settings: - Averaging - Rate	<ACQUISITION Average="122" Rate="125"/>
Photodetector: - Responsivity in water - Responsivity in air - Units	<PHOTODETECTOR ResponsivityW="0.795" ResponsivityA="0.995" Units="uE/(cm2 s)"/>
Microradiometer: - Gain ration High-Medium - Gain ration Medium-Low - Offsets High/Medium/Low	<MICRORADIOMETER GainHM="217.7360077" GainML="198.2780151" OffsetH="8388592" OffsetM="8388840" OffsetL="8388840"/>

[SENSOR_HYDROC]

Information	Format
Identification: - Serial number	<SENSOR SN=”XXXXX”/>
Sensor board identification: - Firmware version - Hardware version	<HYDROC_BOARD Firmware=”xxxxx” Hardware=”xxxxx”/>

[SENSOR_IMU]

Information	Format
Identification: - Serial number - Orientation - Mode	<SENSOR SN=”XXXXX” Orientation=”X” Mode=”X”/>
Temperature correction: - Offset	<TEMPERATURE t0=”xxx”
Accelerometer correction: - Offset - Gain	<ACCELEROMETER ax0=”xxx” ay0=”xxx” az0=”xxx” axg=”x.xxx” ayg=”x.xxx” azg=”x.xxx”/>
Gyroscope correction: - Offset	<GYROSCOPE gx0=”xxx” gy0=”xxx” gz0=”xxx”/>
Magnetometer correction: - Offset	<MAGNETOMETER mx0=”xxx” my0=”xxx” mz0=”xxx”/>
Compass correction: - Hard-iron offset - Soft-iron matrix	<COMPASS hi1=”xxx” hi2=”xxx” si11=”x.xxx” si12=”x.xxx” si21=”x.xxx” si22=”x.xxx”/>

[SENSOR_FLTRIDER]

Information	Format
Identification: - Serial number	<SENSOR SN=”XXXXX”/>
Sensor board identification:	<FLTRIDER_BOARD Firmware=”xxxxx”/>

- Firmware version	

[SENSOR_PAL]

Information	Format
Identification: - Serial number - Model	<SENSOR SN=”XXXXX” Model=”xxx”/>
Sensor board identification: - Firmware version	<PAL_BOARD Firmware=”xxxxx”/>
Acoustic settings: - Filter - Gain	<ACOUSTIC Filter="10" Gain="11.9"/>
Hydrophone identification: - Serial number - Model - Sensitivity	<HYDROPHONE SN="xxx" Model="HTI-96-MIN" Sensitivity="-175.6"/>
Spectrum output identification: - Bands number - Bands	<SPECTRUM Length="9" Bands="63.0,125.0,400.0,1000.0,2000.0,5000.0,8000.0,13000.0,21000.0"/>
Rain quantification: - Coefficient “a3” - Coefficient “a2” - Coefficient “a1” - Coefficient “a0”	<QUANTIFY_WIND a3="0.0009" a2="-0.0984" a1="3.7163" a0="-45.5340"/>
Wind quantification: - Coefficient “b3” - Coefficient “b2” - Coefficient “b1” - Coefficient “b0”	<QUANTIFY_RAIN b3="0.0023" b2="-0.3280" b1="15.8386" b0="-255.6015"/>
Noise quantification: - Method	<CLASSIFY_NOISE c0="0"/>

[SENSOR_TRIDENTE]

Information	Format
Identification: - Serial number	<SENSOR SN=”XXXXX”/>
Output configuration: - Sampling regime - Average number - Output number	<CONFIGURATION Mode="Aggregate" Average="32" Outputs="3"/>

[SENSOR_SENSO]

Information	Format
Identification: - Serial number - Model	<SENSOR SN=”XXXXX” Model=”SENSO3”/>
Channel configuration: - Identification string “na” - Calibration type “ca” - Calibration points “rx/cx”	<CHANNEL_xx na="Chlorophyll" ca="M" r0="0.0" c0="0.000" r1="34.7" c1="35.098" r2="54.8" c2="55.453"/>

[SENSOR_RINKO]

Information	Format
Identification: - Serial number - Model	<SENSOR SN=”XXXXX” Model=”ARO-FT”/>
Sensor board identification: Firmware version	<RINKO_BOARD Firmware="XXXXX"/>
Temperature calibration: - Coefficient “c0” to “c2” - Coefficients “d0” to “d4” - Coefficient “e0”	<TEMPERATURE_COEFF c0="0.0009" c1="-0.0984" c2="3.7163" d0="-45.5340" d1="0.0009" d2="-0.0984" d3="3.7163" d4="15.8386" e0="0.0023"/>
Oxygen calibration: - Coefficients “A” to “H”	<OXYGEN_COEFF A="0.0023" B="-0.3280" C="15.8386" D="-255.6015" E="3.7163" F="-45.5340" G="-0.3280" H="15.8386"/>
Pressure calibration: - Coefficient “Cp”	<PRESSURE_COEFF Cp="0.0023"/>

Example:

```
<FLOAT>
    <PROFILER SN="AABBCC-DDEEFFF" Model="PROVOR-V"/>
    <TELECOM Type="IRIDIUM" CID="8988169224001045237"/>
    <HARDWARE>
        <CONTROL_BOARD Model="APMT" Firmware="1.07.019"/>
        <MEASURE_BOARD Model="USEA" Firmware="1.00.019"/>
    </HARDWARE>
    <SENSORS>
        <SENSOR_UVP6>
            <SENSOR SN="000001HF" Model="UVP6-LP"/>
            <HW_CONF frame="000001HF,0,ACQ_NKE_00h,0,001,1,150,250,,999.000,393819,10000,2,192.1
            <ACQ_CONF_01 frame="ACQ_NKE_00H,0,1.000,1,1,0,0,1,1,10,0,500,1.0,10,10,0,1000,0,40,1
            <ACQ_CONF_02 frame="ACQ_NKE_00L,0,1.000,1,1,0,0,1,1,10,0,500,1.0,100,10,0,1000,0,40,
            <ACQ_CONF_03 frame="ACQ_NKE_01H,0,1.000,1,1,0,0,1,1,10,0,500,1.0,10,10,0,1000,0,40,1
            <ACQ_CONF_04 frame="ACQ_NKE_01L,0,1.000,1,1,0,0,1,1,10,0,500,1.0,100,10,0,1000,0,40,
            <ACQ_CONF_05 frame="ACQ_NKE_20H,0,1.000,1,1,0,0,1,1,10,2,500,1.0,10,10,0,1000,0,40,1
            <ACQ_CONF_06 frame="ACQ_NKE_20L,0,1.000,1,1,0,0,1,1,10,2,500,1.0,100,10,0,1000,0,40,
            <ACQ_CONF_07 frame="ACQ_NKE_21H,0,1.000,1,1,0,0,1,1,10,2,500,1.0,10,10,0,1000,0,40,1
            <ACQ_CONF_08 frame="ACQ_NKE_21L,0,1.000,1,1,0,0,1,1,10,2,500,1.0,100,10,0,1000,0,40,
            <ACQ_CONF_09 frame="ACQ_NKE_CUST_1,0,1.000,1,1,0,0,1,1,10,2,500,1.0,15,10,0,1000,0,4
            <ACQ_CONF_10 frame="ACQ_NKE_CUST_2,0,1.000,1,1,0,0,1,1,10,1,500,1.0,30,10,0,1000,0,4
        </SENSOR_UVP6>
        <SENSOR_DO>
            <SENSOR SN="03014" Model="DO4330"/>
            <PHASE_COEFF c0="9.500000e-02"/>
            <SVU_FOIL_COEFF c0="2.749372e-03" c1="1.217100e-04" c2="2.062590e-06" c3="1.655341e+
        </SENSOR_DO>
        <SENSOR_OCR>
            <SENSOR SN="00184" Model="OCR504"/>
            <CHANNEL_01 a0="2.14766538910e+09" a1="1.63699685504e-07" im="1.161e+00"/>
            <CHANNEL_02 a0="2.14754852610e+09" a1="2.00131764196e-07" im="1.368e+00"/>
            <CHANNEL_03 a0="2.14742382370e+09" a1="2.04019168366e-07" im="1.365e+00"/>
            <CHANNEL_04 a0="2.14745155870e+09" a1="3.03210835119e-06" im="1.359e+00"/>
        </SENSOR_OCR>
        <SENSOR_ECO>
            <SENSOR SN="02345" Model="ECO3"/>
            <CHANNEL_01 sf="7.200e-03" dc="47"/>
            <CHANNEL_02 sf="2.029e-06" dc="43"/>
            <CHANNEL_03 sf="8.720e-02" dc="30"/>
        </SENSOR_ECO>
        <SENSOR_SUNA>
            <SENSOR SN="00555" Model="SUNA-V2" Spectrum="Output pixels 38-77"/>
            <SUNA_BOARD Firmware="2.2.13"/>
            <SPECTROMETER spintper="400"/>
        </SENSOR_SUNA>
        <SENSOR_SBE41>
            <SENSOR SN="08959" Model="SBE41-CP"/>
            <SENSOR_PRESSURE SN="004978241"/>
        </SENSOR_SBE41>
    </SENSORS>
</FLOAT>
```

6. Sensor data files

The system can generate sensor data files under three formats:

- Text format (*.csv)
- Standard binary format (*.hex)
- Extended binary format (*.hex)

6.1. Transmission of sensor data

The various sensor data files generated are named using the format "xxxx_ccc_mm_ssssss.ext", with:

- "xxxx": float's hexadecimal serial number (4 characters)
- "ccc": cycle number (3 characters)
- "mm": pattern number (2 characters)
- "sssss": sensor ID (N characters, e.g. "sbe41")
- "ext": file extension

All data from a sensor are saved in a unique data file associated to the cycle/pattern.

6.2. ID tags

All files are organised in sections using ID tags for the phase and processing type.

6.2.1. Navigation phase identification

At every change of navigation phase, an ID string is inserted. The various ASCII strings are as follows:

- **[DESCENT]**: Data from the descent phase, from the surface to the drift depth
- **[PARK]**: Data from the drift phase
- **[DEEP_PROFILE]**: Data from the descent phase, from the drift depth to the measurement depth
- **[SHORT_PARK]**: Data from the drift phase, from the drift depth to the measurement depth
- **[ASCENT]**: Data from the ascent phase
- **[SURFACE]**: Data from the surface phase

6.2.2. Processing type identification

At every change of phase or processing area, an ID string for the processing type is inserted. The various ASCII strings are as follows:

- **(RW)**: For raw data
- **(DW)**: For decimated raw data
- **(AM)**: For arithmetic mean
- **(SD)**: For standard deviation
- **(MD)**: For median
- **(SS)**: For subsurface point

Several processing types can follow one another. The combinations are as follows:

- (RW)
- (DW)
- (AM)
- (AM)(SD)
- (AM)(MD)
- (AM)(SD)(MD)
- (SS)

6.3.Recording in text format

Spreadsheet format with the possibility of choosing the decimal separator, the column separator as well as the timestamping format can be selected.

```
[DESCENT]
(AM) (SD) (MD)
2015-06-15 08:34:51;4.90;17.4640;35.798;0.0000;0.0000;4.90;17.4640;35.798
2015-06-15 08:34:51;5.50;17.4600;35.797;0.0030;0.0000;5.50;17.4600;35.797
2015-06-15 08:34:51;6.40;17.4530;35.797;0.0030;0.0000;6.40;17.4530;35.797
2015-06-15 08:34:51;7.40;17.4470;35.796;0.0030;0.0000;7.40;17.4500;35.797
2015-06-15 08:34:51;8.50;17.4380;35.796;0.0030;0.0000;8.50;17.4360;35.796
2015-06-15 08:34:51;9.50;17.4320;35.796;0.0030;0.0000;9.50;17.4290;35.796
```

6.4.Recording in binary format

6.4.1. Encoding format

The data is saved in "little endian" format with byte alignment objects:

- **BYTE** - char (1 byte)
- **SHORTINT** - short integer (2 bytes)
- **LONGINT** - long integer (4 bytes)
- **EPOCH** – absolute timestamping (4 bytes) - Unix Epoch format - January 1st, 1970 at 00:00
- **FLOAT** - floating (4 bytes - float) – IEEE-754 format

6.4.2. File structure

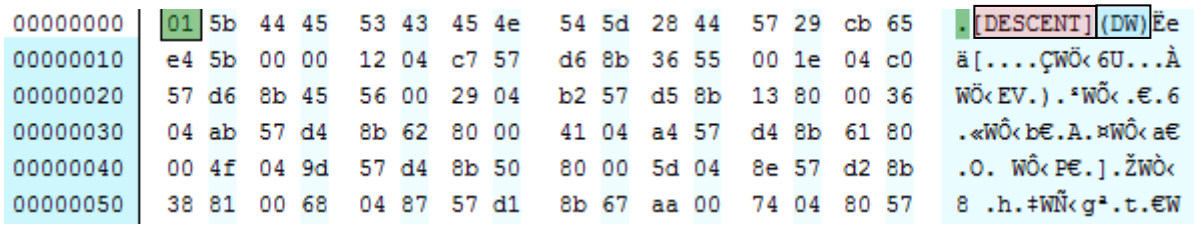
Each file begins with an encoding identifying byte:

- 01: SBE41 sensor - Extended format
- 02: SBE41 sensor - Standard format
- 03: DO sensor
- 04: OCR504 sensor (4 channels)
- 05: OCR507 sensor (7 channels)
- 06: OCR507-IR sensor (14 channels)
- 07: ECO1 sensor (1 channel)
- 08: ECO2 sensor (2 channels)
- 09: ECO3 sensor (3 channels)
- 10: CROVER sensor
- 11: SBEPH sensor
- 12: SUNA sensor (45 outputs spectrum)
- 13: SUNA sensor (90 outputs spectrum)
- 14: UVP6 sensor (LPM data)
- 15: None
- 16: None
- 17: UVP6 sensor (BLACK data)
- 18: ECO1v2 sensor (1 channel)
- 19: ECO2v2 sensor (2 channels)
- 20: ECO3v2 sensor (3 channels)
- 21: ECO4v2 sensor (4 channels)
- 22: EXTTRIG sensor
- 23: RAMSES sensor
- 24: OPUS sensor (LIGHT data)
- 25: OPUS sensor (BLACK data)
- 26: MPE sensor
- 27: UVP6 sensor (LPMv2 data)
- 28: UVP6 sensor (TAXOv2 data)
- 29: UVP sensor (BLACKv2 data)
- 30: HYDROC sensor (M data)
- 31: HYDROC sensor (C data)

- 32: RAMSES2 sensor
- 33: IMU sensor (raw data)
- 34: IMU sensor (tilt plus heading data)
- 35: WAVE sensor (raw data)
- 36: PAL sensor (rain plus wind format)
- 37: PAL sensor (high resolution, meteorological spectrum format)
- 38: PAL sensor (high dynamic range, meteorological spectrum format)
- 39: PAL sensor (full spectrum format)
- 40: RAMSES1v2 sensor
- 41: RAMSES2v2 sensor
- 42: OPUS sensor (LIGHTv2 data)
- 43: OPUS sensor (BLACKv2 data)
- 44: TRIDENTE3 sensor (3 channels)
- 45: TRIDENTE9 sensor (9 channels)
- 46: SENSO2 sensor (2 channels)
- 47: SENSO3 sensor (3 channels)
- 48: SENSO6 sensor (6 channels)
- 49: SENSO9 sensor (9 channels)
- 50: RINKO sensor

This is followed by groups of recordings identified by the tags.

Example: SBE41 standard format file, descent phase



Note: In Iridium RUDICS transmission, files can be supplemented by padding bytes (0x1A). These bytes must not be decoded. Each file can end with a sequence of 0 to 1023 padding bytes.

6.4.3. Timestamping

Records can be timestamped with absolute timestamps (Unix Epoch format) or relative timestamps since a reference date.

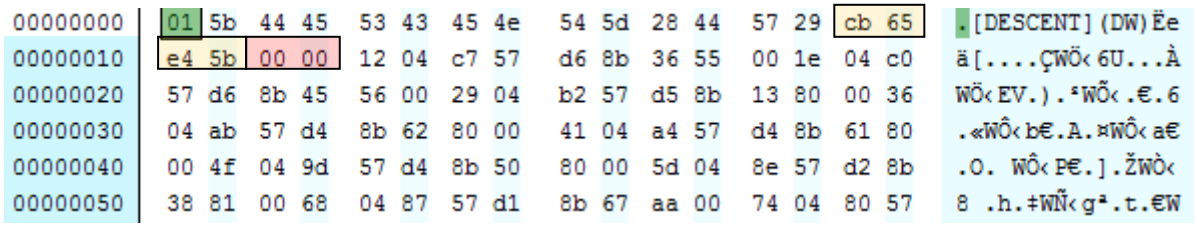
Absolute timestamping is used for raw data records in parking phase (where the delay between records can be greater than hours). Data is encoded as follows:

- **Timestamp** absolute encoding (EPOCH)

Relative timestamping is used for the others records. The reference date is saved after the navigation phase tag. The following records use an offset (count in seconds) from this reference time. Data is encoded as follows:

- **Timestamp** offset in seconds (unsigned integer) – code = value - reference time

Example: Relative timestamping encoding



Timestamp = 1541694923 (0x5BE465CB) + 0 sec (0x0000) => 2018-11-08 16:35:23

6.5.SBE41 sensor

6.5.1. Data encoding

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Temperature** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0
- **Salinity** in mpsu (unsigned integer) – code = value * 1000.0
- **Temperature standard deviation** in m°C (signed integer) – code = value * 1000.0
- **Salinity standard deviation** in mpsu (signed integer) – code = value * 1000.0

6.5.2. Standard binary format

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The recording structures are as follows:

Subsurface - (SS)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned short int uiATemperature;
  unsigned short int uiSalinity;
 }
 tSSStd;
```

Subsurface data encoding (ascent)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Subsurface record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
				10

Raw data (drift) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
 }
 tRStdPark;
```

Raw data encoding (park phase)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
				10

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
 }
 tRStdNav;
```

Raw data encoding (navigation phase)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
				8

Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
 }
 tMStd;
```

Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
				8

Averaged data and standard deviation – (AM)(SD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  signed char cStdDeviationTemperature;
  signed char cStdDeviationSalinity;
 }
 tMECStd;
```

Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Standard deviation record				
Temperature STD	BYTE	-0.128 to +0.127°C	0.001°C	1
Salinity STD	BYTE	-0.128 to +0.127 psu	0.001 psu	1
				10



Averaged data and median – (AM)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  unsigned short int uiMedianPressure;
  unsigned short int uiMedianTemperature;
  unsigned short int uiMedianSalinity;
 }
 tMMStd;
```

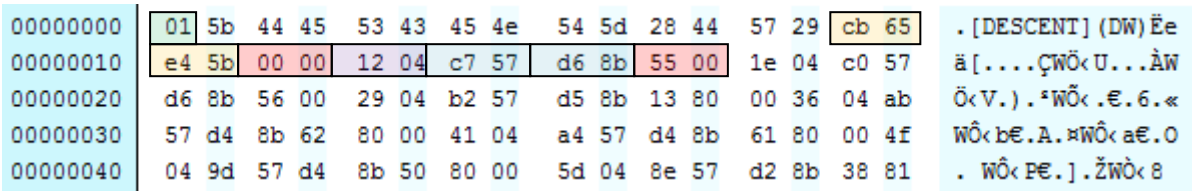
Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
				14

Averaged data, standard deviation and median – (AM)(SD)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  signed char cStdDeviationTemperature;
  signed char cStdDeviationSalinity;
  unsigned short int uiMedianPressure;
  unsigned short int uiMedianTemperature;
  unsigned short int uiMedianSalinity;
 }
 tMECMStd;
```

Average + standard deviation + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Standard deviation record				
Temperature STD	BYTE	-0.128 to +0.127°C	0.001°C	1
Salinity STD	BYTE	-0.128 to +0.127 psu	0.001 psu	1
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
				16

Example:
[DESCENT]
(DW)
2018-11-08 16:35:23,4.2,17.471,35.798
2018-11-08 16:36:48,5.4,17.464,35.798



Timestamp 1 = 1541694923 (0x5BE465CB) + 0 sec (0x0000) => 2018-11-08 16:35:23
Pressure 1 = 1042 (0x0412) => ((1042 / 10.0) – 100.0) = 4.2 dbar
Temperature 1 = 22471 (0x57C7) => ((22471 / 1000.0) – 5.0) = 17.471°C
Salinity 1 = 35798 (0xD68B) => (35798 / 1000.0) = 35.798 psu
Timestamp 2 = Timestamp 1 + 85 sec (0x0055) => 2018-11-08 16:36:48
…

6.5.3. Extended binary format

Advantages of this format compared to the standard version:

- Improved resolution of recordings (pressure and temperature)

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The resolution extension for pressure and temperature measurements is encoded on a shared byte using masks, as follows:

- Pressure extension (0.01 dbar) = value & 0xF0
- Temperature extension (0.1 m°C) = value & 0x0F

The available recording structures are as follows:

Subsurface – (SS)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned short int uiTemperature;
  unsigned short int uiSalinity;
  unsigned char ucPTExtra;
 }
 tSSExt;
```

Subsurface data encoding (ascent)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Subsurface record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1
				11

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  unsigned char ucAveragePTExtra;
 }
 tRExtPark;
```

Raw data encoding (park phase)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1
				11

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  unsigned char ucAveragePTExtra;
 }
 tRExtNav;
```

Raw data encoding (navigation phase)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1
				9

Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  unsigned char ucAveragePTExtra;
 }
 tMExt;
```

Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1
				9

Averaged data and standard deviation – (AM)(SD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  unsigned char ucAveragePTExtra;
  signed char cStdDeviationTemperature;
  signed char cStdDeviationSalinity;
 }
 tMECExt;
```

Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1

Standard deviation record				
Temperature STD	BYTE	-0.128 to +0.127°C	0.001°C	1
Salinity STD	BYTE	-0.128 to +0.127 psu	0.001 psu	1
				11

Averaged data and median – (AM)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  unsigned char ucAveragePTExtra;
  unsigned short int uiMedianPressure;
  unsigned short int uiMedianTemperature;
  unsigned short int uiMedianSalinity;
  unsigned char ucMedianPTExtra;
 }
 tMMExt;
```

Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1
				16

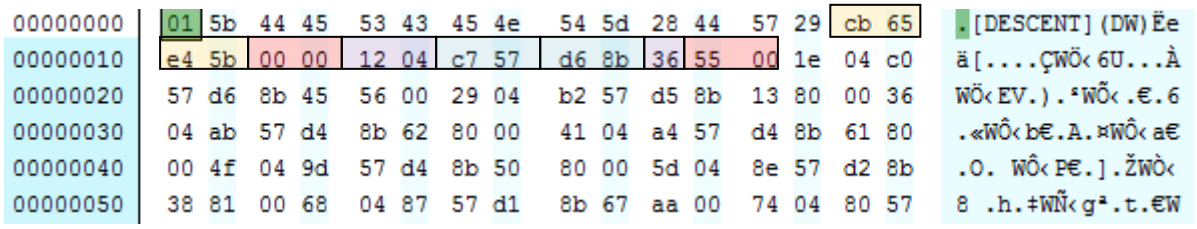
Averaged data, standard deviation and median – (AM)(SD)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiAverageTemperature;
  unsigned short int uiAverageSalinity;
  unsigned char ucAveragePTExtra;
  signed char cStdDeviationTemperature;
  signed char cStdDeviationSalinity;
  unsigned short int uiMedianPressure;
  unsigned short int uiMedianTemperature;
  unsigned short int uiMedianSalinity;
  unsigned char ucMedianPTExtra;
 }
 tMECMExt;
```

Average + standard deviation + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1
Standard deviation record				
Temperature STD	BYTE	-0.128 to +0.127°C	0.001°C	1
Salinity STD	BYTE	-0.128 to +0.127 psu	0.001 psu	1
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Pressure extra Temperature extra	BYTE_H BYTE_L	0.00 to 0.09 dbar 0.0000 to 0.0009°C	0.01 dbar 0.0001°C	1
				18

Example:

[DESCENT]
(DW)
2018-11-08 16:35:23,4.23,17.4716,35.798
2018-11-08 16:36:48,5.44,17.4645,35.798



Timestamp 1 = 1541694923 (0x5BE465CB) + 0 sec (0x0000) => 2018-11-08 16:35:23
Pressure 1 = 1042 (0x0412) + 0.03 dbar (0x3_) => ((1042 / 10.0) – 100.0) + 0.03 = 4.23 dbar
Temperature 1 = 22471 (0x57C7) + 0.0006°C (0x_6) => ((22471 / 1000.0) – 5.0) + 0.006 = 17.4716°C
Salinity 1 = 35798 (0xD68B) => (35798 / 1000.0) = 35.798 psu
Timestamp 2 = Timestamp 1 + 85 sec (0x0055) => 2018-11-08 16:36:48
…

6.6.DO sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Phase** in degrees (float) – code = value
- **Temperature** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0
- **Phase standard deviation** in degrees (signed integer) – code = value * 1000.0
- **Temperature standard deviation** in m°C (signed integer) – code = value * 1000.0

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  float fC1RPhase;
  float fC2RPhase;
  unsigned short int uiTemperature;
 }
 tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
C1 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
C2 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
				16

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  float fC1RPhase;
  float fC2RPhase;
  unsigned short int uiTemperature;
 }
 tRNav;
```

Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
C1 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
C2 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
				14

Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  float fAverageC1RPhase;
  float fAverageC2RPhase;
  unsigned short int uiAverageTemperature;
 }
 tM;
```

Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
C1 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
C2 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
				14

Averaged data and standard deviation – (AM)(SD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  float fAverageC1RPhase;
  float fAverageC2RPhase;
  unsigned short int uiAverageTemperature;
  signed short int iStdDeviationC1RPhase;
  signed short int iStdDeviationC2RPhase;
  signed char cStdDeviationTemperature;
 }
 tMEC;
```

Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
C1 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
C2 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Standard deviation record				
C1 Phase	SHORTINT	-32.768 to +32.767 deg	0.001 deg	2
C2 Phase	SHORTINT	-32.768 to +32.767 deg	0.001 deg	2
Temperature	BYTE	-0.128 to +0.127°C	0.001°C	1
				19

Averaged data and median – (AM)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  float fAverageC1RPhase;
  float fAverageC2RPhase;
  unsigned short int uiAverageTemperature;
  unsigned short int uiMedianPressure;
  float fMedianC1RPhase;
  float fMedianC2RPhase;
  unsigned short int uiMedianTemperature;
 }
 tMM;
```

Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
C1 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
C2 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
C1 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
C2 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
				26

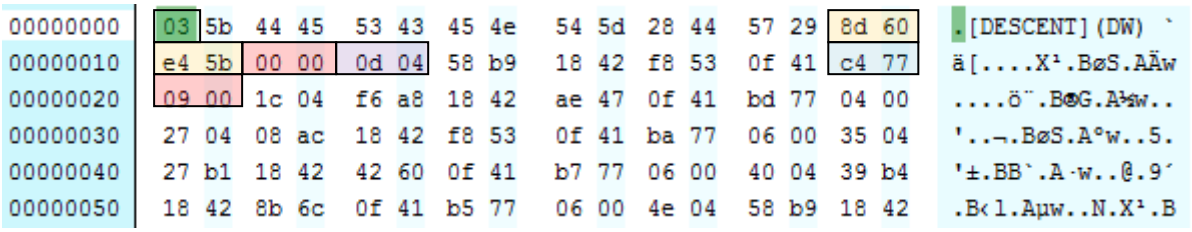
Averaged data, standard deviation and median – (AM)(SD)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  float fAverageC1RPhase;
  float fAverageC2RPhase;
  unsigned short int uiAverageTemperature;
  signed short int iStdDeviationC1RPhase;
  signed short int iStdDeviationC2RPhase;
  signed char cStdDeviationTemperature;
  unsigned short int uiMedianPressure;
  float fMedianC1RPhase;
  float fMedianC2RPhase;
  unsigned short int uiMedianTemperature;
 }
 tMECM;
```

Average + standard deviation + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
C1 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
C2 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Standard deviation record				
C1 Phase	SHORTINT	-32.768 to +32.767 deg	0.001 deg	2
C2 Phase	SHORTINT	-32.768 to +32.767 deg	0.001 deg	2
Temperature	BYTE	-0.128 to +0.127°C	0.001°C	1
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
C1 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
C2 Phase	FLOAT	-90.0 to +90.0 deg	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
				31

Example:

[DESCENT]
(DW)
2018-11-08 16:13:01,3.7,38.181,8.958,25.66
2018-11-08 16:13:10,5.2,38.165,8.955,25.65



Timestamp 1 = 1541693581 (0x5BE4608D) + 0 sec (0x0000) => 2018-11-08 16:13:01
Pressure 1 = 1037 (0x040D) => ((1037 / 10.0) – 100.0) = 3.7 dbar
[...]
Temperature 1 = 30660 (0x77C4) => ((30660 / 1000.0) – 5.0) = 25.66°C
Timestamp 2 = Timestamp 1 + 9 sec (0x0009) => 2018-11-08 16:13:10
…

6.7.OCR sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Channel count** (unsigned long int) – code = value
- **Channel count standard deviation** (signed long int) – code = value

Data record can be composed of counts from 4, 7 or 14 channels (depending on the sensor's configuration).

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned long int uliCountChannel1;
  unsigned long int uliCountChannel2;
  [...]
  unsigned long int uliCountChannelN;
 }
 tRPark;
```

4 channels - Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	LONGINT	0 to +4294967295	1	4
Channel 2 count	LONGINT	0 to +4294967295	1	4
Channel 3 count	LONGINT	0 to +4294967295	1	4
Channel 4 count	LONGINT	0 to +4294967295	1	4
				22

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  unsigned long int uliCountChannel1;
  unsigned long int uliCountChannel2;
  [...]
  unsigned long int uliCountChannelN;
 }
 tRNav;
```

4 channels - Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	LONGINT	0 to +4294967295	1	4
Channel 2 count	LONGINT	0 to +4294967295	1	4
Channel 3 count	LONGINT	0 to +4294967295	1	4
Channel 4 count	LONGINT	0 to +4294967295	1	4
				20

Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned long int uliAverageCountChannel1;
  unsigned long int uliAverageCountChannel2;
  [...]
  unsigned long int uliAverageCountChannelN;
 }
 tM;
```

4 channels - Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	LONGINT	0 to +4294967295	1	4
Channel 2 count	LONGINT	0 to +4294967295	1	4
Channel 3 count	LONGINT	0 to +4294967295	1	4
Channel 4 count	LONGINT	0 to +4294967295	1	4
				20

Averaged data and standard deviation – (AM)(SD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned long int uliAverageCountChannel1;
  unsigned long int uliAverageCountChannel2;
  [...]
  unsigned long int uliAverageCountChannelN;
  signed long int liStdDeviationCountChannel1;
  signed long int liStdDeviationCountChannel2;
  [...]
  signed long int liStdDeviationCountChannelN;
 }
 tMEC;
```

4 channels - Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	LONGINT	0 to +4294967295	1	4
Channel 2 count	LONGINT	0 to +4294967295	1	4
Channel 3 count	LONGINT	0 to +4294967295	1	4
Channel 4 count	LONGINT	0 to +4294967295	1	4
Standard deviation record				
Channel 1 count	LONGINT	-2147483648 to +2147483647	1	4
Channel 2 count	LONGINT	-2147483648 to +2147483647	1	4
Channel 3 count	LONGINT	-2147483648 to +2147483647	1	4
Channel 4 count	LONGINT	-2147483648 to +2147483647	1	4
				34

Averaged data and median – (AM)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned long int uliAverageCountChannel1;
  unsigned long int uliAverageCountChannel2;
  [...]
  unsigned long int uliAverageCountChannelN;
  unsigned short int uiMedianPressure;
  unsigned long int uliMedianCountChannel1;
  unsigned long int uliMedianCountChannel2;
  [...]
  unsigned long int uliMedianCountChannelN;
 }
 tMM;
```

4 channels - Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	LONGINT	0 to +4294967295	1	4
Channel 2 count	LONGINT	0 to +4294967295	1	4
Channel 3 count	LONGINT	0 to +4294967295	1	4
Channel 4 count	LONGINT	0 to +4294967295	1	4
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	LONGINT	0 to +4294967295	1	4
Channel 2 count	LONGINT	0 to +4294967295	1	4
Channel 3 count	LONGINT	0 to +4294967295	1	4
Channel 4 count	LONGINT	0 to +4294967295	1	4
				38

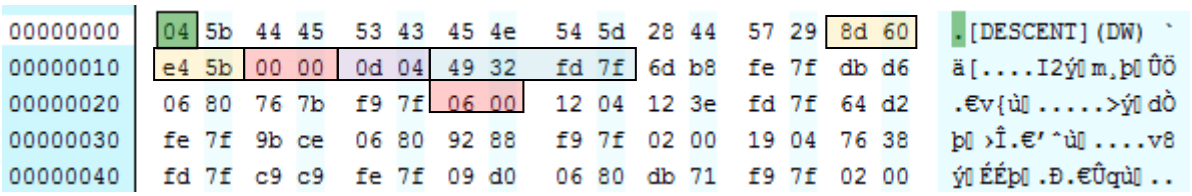
Averaged data, standard deviation and median – (AM)(SD)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned long int uliAverageCountChannel1;
  unsigned long int uliAverageCountChannel2;
  [...]
  unsigned long int uliAverageCountChannelN;
  signed long int liStdDeviationCountChannel1;
  signed long int liStdDeviationCountChannel2;
  [...]
  signed long int liStdDeviationCountChannelN;
  unsigned short int uiMedianPressure;
  unsigned long int uliMedianCountChannel1;
  unsigned long int uliMedianCountChannel2;
  [...]
  unsigned long int uliMedianCountChannelN;
 }
 tMECM;
```

4 channels - Average + standard deviation + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	LONGINT	0 to +4294967295	1	4
Channel 2 count	LONGINT	0 to +4294967295	1	4
Channel 3 count	LONGINT	0 to +4294967295	1	4
Channel 4 count	LONGINT	0 to +4294967295	1	4
Standard deviation record				
Channel 1 count	LONGINT	-2147483648 to +2147483647	1	4
Channel 2 count	LONGINT	-2147483648 to +2147483647	1	4
Channel 3 count	LONGINT	-2147483648 to +2147483647	1	4
Channel 4 count	LONGINT	-2147483648 to +2147483647	1	4
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	LONGINT	0 to +4294967295	1	4
Channel 2 count	LONGINT	0 to +4294967295	1	4
Channel 3 count	LONGINT	0 to +4294967295	1	4
Channel 4 count	LONGINT	0 to +4294967295	1	4
				54

Example:

[DESCENT]
(DW)
2018-11-08 16:13:01,3.7,2147299913,2147399789,2147931867,2147056502
2018-11-08 16:13:07,4.2,2147302930,2147406436,2147929755,2147059858



Timestamp 1 = 1541693581 (0x5BE4608D) + 0 sec (0x0000) => 2018-11-08 16:13:01
Pressure 1 = 1037 (0x040D) => ((1037 / 10.0) – 100.0) = 3.7 dbar
Count 1 = 2147299913 (0x7FFD3249)
[...]
Timestamp 2 = Timestamp 1 + 6 sec (0x0006) => 2018-11-08 16:13:07
…

6.8.ECO sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Channel count** (signed short int) – code = value
- **Channel count standard deviation** (signed char) – code = value

Data record can be composed of counts from 1 to 3 channels (depending on the sensor's configuration).

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW) or (DW)

```
struct
{
 unsigned long int uliDateTime;
 unsigned short int uiPressure;
 signed short int iCountChannel1;
[...]
 signed short int iCountChannelN;
}
 tRPark;
```

3 channels - Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	-64 to +4130	1	2
Channel 2 count	SHORTINT	-64 to +4130	1	2
Channel 3 count	SHORTINT	-64 to +4130	1	2
				12

Raw data (navigation) – (RW) or (DW)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiPressure;
 signed short int iCountChannel1;
[...]
 signed short int iCountChannelN;
}
 tRNav;
```

3 channels - Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	-	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	-64 to +4130	1	2
Channel 2 count	SHORTINT	-64 to +4130	1	2
Channel 3 count	SHORTINT	-64 to +4130	1	2
				10

Averaged data – (AM)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiAveragePressure;
 signed short int iAverageCountChannel1;
[...]
 signed short int iAverageCountChannelN;
}
 tM;
```

3 channels - Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	-64 to +4130	1	2
Channel 2 count	SHORTINT	-64 to +4130	1	2
Channel 3 count	SHORTINT	-64 to +4130	1	2
				10

Averaged data and standard deviation – (AM)(SD)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiAveragePressure;
 signed short int iAverageCountChannel1;
[...]
 signed short int iAverageCountChannelN;
 signed char cStdDeviationCountChannel1;
[...]
 signed char cStdDeviationCountChannelN;
}
 tMEC;
```

3 channels - Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	-64 to +4130	1	2
Channel 2 count	SHORTINT	-64 to +4130	1	2
Channel 3 count	SHORTINT	-64 to +4130	1	2
Standard deviation record				
Channel 1 count	BYTE	-128 to +127	1	1
Channel 2 count	BYTE	-128 to +127	1	1
Channel 3 count	BYTE	-128 to +127	1	1
				13

Averaged data and median – (AM)(MD)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiAveragePressure;
 signed short int iAverageCountChannel1;
[...]
 signed short int iAverageCountChannelN;
 unsigned short int uiMedianPressure;
 signed short int iMedianCountChannel1;
[...]
 signed short int iMedianCountChannelN;
}
 tMM;
```

3 channels - Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	-64 to +4130	1	2
Channel 2 count	SHORTINT	-64 to +4130	1	2
Channel 3 count	SHORTINT	-64 to +4130	1	2
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	-64 to +4130	1	2
Channel 2 count	SHORTINT	-64 to +4130	1	2
Channel 3 count	SHORTINT	-64 to +4130	1	2
				18

Averaged data, standard deviation and median – (AM)(SD)(MD)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiAveragePressure;
 signed short int iAverageCountChannel1;
 [...]
 signed short int iAverageCountChannelN;
 signed char cStdDeviationCountChannel1;
 [...]
 signed char cStdDeviationCountChannelN;
 unsigned short int uiMedianPressure;
 signed short int iMedianCountChannel1;
 [...]
 signed short int iMedianCountChannelN;
}
 tMECM;
```

3 channels - Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	-64 to +4130	1	2
Channel 2 count	SHORTINT	-64 to +4130	1	2
Channel 3 count	SHORTINT	-64 to +4130	1	2
Standard deviation record				
Channel 1 count	BYTE	-128 to +127	1	1
Channel 2 count	BYTE	-128 to +127	1	1
Channel 3 count	BYTE	-128 to +127	1	1
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	-64 to +4130	1	2
Channel 2 count	SHORTINT	-64 to +4130	1	2
Channel 3 count	SHORTINT	-64 to +4130	1	2
				21

Example:
[DESCENT]
(DW)
2018-11-08 16:13:01,3.7,4130,4130,4130
2018-11-08 16:13:07,4.2,4130,4130,4130

```
00000000  09 5b 44 45 53 43 45 4e  54 5d 28 44 57 29 8d 60  .[DESCENT](DW) `
00000010  e4 5b 00 00 0d 04 22 10  22 10 22 10 06 00 12 04  ä[....".".".....
00000020  22 10 22 10 22 10 02 00  19 04 22 10 22 10 22 10  ".".".....".".".
00000030  02 00 1f 04 22 10 22 10  22 10 02 00 23 04 22 10  ....".".".."#.".
00000040  22 10 22 10 02 00 2a 04  22 10 22 10 22 10 02 00  ".".."*.".".".."
```

Timestamp 1 = 1541693581 (0x5BE4608D) + 0 sec (0x0000) => 2018-11-08 16:13:01
Pressure 1 = 1037 (0x040D) => ((1037 / 10.0) – 100.0) = 3.7 dbar
Count 1 = 4130 (0x1022)
[...]
Timestamp 2 = Timestamp 1 + 6 sec (0x0006) => 2018-11-08 16:13:07
…

6.9.ECOv2 sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Channel count** (unsigned short int) – code = value
- **Channel count standard deviation** (signed int) – code = value

Data record can be composed of counts from 1 to 4 channels (depending on the sensor's configuration).

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW) or (DW)

```
struct
{
 unsigned long int uliDateTime;
 unsigned short int uiPressure;
 unsigned short int uiCountChannel1;
[...]
 unsigned short int uiCountChannelN;
}
 tRPark;
```

3 channels - Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65000	1	2
Channel 2 count	SHORTINT	0 to +65000	1	2
Channel 3 count	SHORTINT	0 to +65000	1	2
				12

Raw data (navigation) – (RW) or (DW)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiPressure;
 unsigned short int uiCountChannel1;
[...]
 unsigned short int uiCountChannelN;
}
 tRNav;
```

3 channels - Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	-	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65000	1	2
Channel 2 count	SHORTINT	0 to +65000	1	2
Channel 3 count	SHORTINT	0 to +65000	1	2
				10

Averaged data – (AM)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiAveragePressure;
 unsigned short int uiAverageCountChannel1;
[...]
 unsigned short int uiAverageCountChannelN;
}
 tM;
```

3 channels - Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65000	1	2
Channel 2 count	SHORTINT	0 to +65000	1	2
Channel 3 count	SHORTINT	0 to +65000	1	2
				10

Averaged data and standard deviation – (AM)(SD)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiAveragePressure;
 unsigned short int uiAverageCountChannel1;
[...]
 unsigned short int uiAverageCountChannelN;
 signed short int iStdDeviationCountChannel1;
[...]
 signed short int iStdDeviationCountChannelN;
}
 tMEC;
```

3 channels - Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65000	1	2
Channel 2 count	SHORTINT	0 to +65000	1	2
Channel 3 count	SHORTINT	0 to +65000	1	2
Standard deviation record				
Channel 1 count	SHORTINT	-32768 to +32767	1	2
Channel 2 count	SHORTINT	-32768 to +32767	1	2
Channel 3 count	SHORTINT	-32768 to +32767	1	2
				16

Averaged data and median – (AM)(MD)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiAveragePressure;
 unsigned short int uiAverageCountChannel1;
[...]
 unsigned short int uiAverageCountChannelN;
 unsigned short int uiMedianPressure;
 unsigned short int uiMedianCountChannel1;
[...]
 unsigned short int uiMedianCountChannelN;
}
 tMM;
```

3 channels - Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65000	1	2
Channel 2 count	SHORTINT	0 to +65000	1	2
Channel 3 count	SHORTINT	0 to +65000	1	2
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65000	1	2
Channel 2 count	SHORTINT	0 to +65000	1	2
Channel 3 count	SHORTINT	0 to +65000	1	2
				18

Averaged data, standard deviation and median – (AM)(SD)(MD)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiAveragePressure;
 unsigned short int uiAverageCountChannel1;
 [...]
 unsigned short int uiAverageCountChannelN;
 signed short int iStdDeviationCountChannel1;
 [...]
 signed short int iStdDeviationCountChannelN;
 unsigned short int uiMedianPressure;
 unsigned short int uiMedianCountChannel1;
 [...]
 unsigned short int uiMedianCountChannelN;
}
 tMECM;
```

3 channels - Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65000	1	2
Channel 2 count	SHORTINT	0 to +65000	1	2
Channel 3 count	SHORTINT	0 to +65000	1	2
Standard deviation record				
Channel 1 count	SHORTINT	-32768 to +32767	1	2
Channel 2 count	SHORTINT	-32768 to +32767	1	2
Channel 3 count	SHORTINT	-32768 to +32767	1	2
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65000	1	2
Channel 2 count	SHORTINT	0 to +65000	1	2
Channel 3 count	SHORTINT	0 to +65000	1	2
				24

6.10. CROVER sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Corrected signal raw count** (signed short int) – code = value
- **Corrected signal raw count standard deviation** (signed short int) – code = value

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  signed short int iCorrectedSignalRawCount;
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
				8

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  signed short int iCorrectedSignalRawCount;
 }
tRNav;
```

Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
				6

Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  signed short int iAverageCorrectedSignalRawCount;
 }
tM;
```

Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
				6

Averaged data and standard deviation – (AM)(SD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  signed short int iAverageCorrectedSignalRawCount;
  signed short int iStdDeviationCorrectedSignalRawCount;
 }
tMEC;
```

Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
Standard deviation record				
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
				8



Averaged data and median – (AM)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  signed short int iAverageCorrectedSignalRawCount;
  unsigned short int uiMedianPressure;
  signed short int iMedianCorrectedSignalRawCount;
 }
tMM;
```

Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Corr Sig Raw	FLOAT	-32768 to +32767	1	2
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
				10

Averaged data, standard deviation and median – (AM)(SD)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  float fAverageBeamAttenuation;
  signed short int iStdDeviationCorrectedSignalRawCount;
  unsigned short int uiMedianPressure;
  signed short int iMedianCorrectedSignalRawCount;
 }
tMECM;
```

Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
Standard deviation record				
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Corr Sig Raw	SHORTINT	-32768 to +32767	1	2
				12

6.11. SBEPH sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Voltage** in µV (signed long int) – code = (value * 1000000.0)
- **Voltage standard deviation** in µV (signed char) – code = (value * 1000000.0)

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  signed long int liRefVoltage;
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	LONGINT	-2500000 to +2500000 µV	1 µV	4
				10

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  signed long int liRefVoltage;
 }
tRNav;
```

Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	LONGINT	-2500000 to +2500000 µV	1 µV	4
				8

Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  signed long int liAverageRefVoltage;
 }
tM;
```

Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	LONGINT	-2500000 to +2500000 µV	1 µV	4
				8

Averaged data and standard deviation – (AM)(SD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  signed long int liAverageRefVoltage;
  signed short int iStdDeviationRefVoltage;
 }
tMEC;
```

Average + standard deviation data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	LONGINT	-2500000 to +2500000 µV	1 µV	4
Standard deviation record				
Voltage	SHORTINT	-32768 to +32767 µV	1 µV	2
				10

Averaged data and median – (AM)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  signed long int liAverageRefVoltage;
  unsigned short int uiMedianPressure;
  signed long int liMedianRefVoltage;
 }
tMM;
```

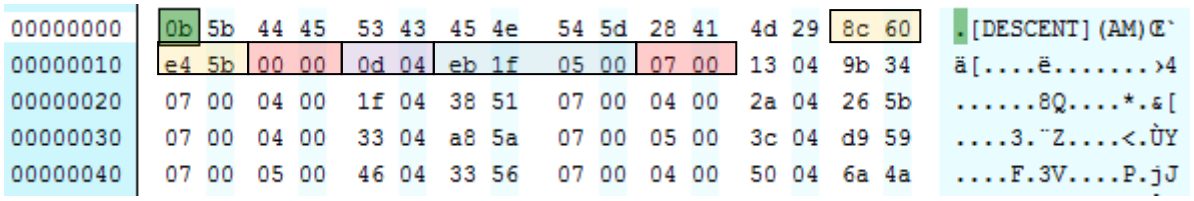
Average + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	LONGINT	-2500000 to +2500000 µV	1 µV	4
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	LONGINT	-2500000 to +2500000 µV	1 µV	4
				14

Averaged data, standard deviation and median – (AM)(SD)(MD)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  signed long int liAverageRefVoltage;
  signed short int iStdDeviationRefVoltage;
  unsigned short int uiMedianPressure;
  signed long int liMedianRefVoltage;
 }
tMECM;
```

Average + standard deviation + median data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	LONGINT	-2500000 to +2500000 µV	1 µV	4
Standard deviation record				
Voltage	SHORTINT	-32768 to +32767 µV	1 µV	2
Median record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	LONGINT	-2500000 to +2500000 µV	1 µV	4
				16

Example:
[DESCENT]
(AM)
2018-11-08 16:13:00,3.7,0.335851
2018-11-08 16:13:07,4.3,0.472219



Timestamp 1 = 1541693580 (0x5BE4608C) + 0 sec (0x0000) => 2018-11-08 16:13:00
Pressure 1 = 1037 (0x040D) => ((1037 / 10.0) – 100.0) = 3.7 dbar
Voltage = 335851 (0x00051FEB) => (335851 / 100000.0) = 0.335851 V
Timestamp 2 = Timestamp 1 + 7 sec (0x0007) => 2018-11-08 16:13:07
…

6.12. SUNA sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Temperature** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0
- **Salinity** in mpsu (unsigned integer) – code = value * 1000.0
- **Humidity** in % (unsigned char) – code = (value * 2.0)
- **Dark mean** (unsigned short int) – code = (value * 10.0)
- **Dark standard deviation** (signed short int) – code = (value * 100.0)
- **Nitrate concentration** in µM (float) – code = value
- **Absorbance fit residual RMS** (float) – code = value
- **Output spectrum** (unsigned short int) – code = value

Data record can be composed of output spectrum from 45 or 90 channels (depending on the sensor's configuration).

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW) or (DW)

```
struct
 {
 unsigned long int uliDateTime;
 struct
 {
  struct
  {
   unsigned short int uiPressure;
   unsigned short int uiTemperature;
   unsigned short int uiSalinity;
  }
  tCTD;
  unsigned short int uiInternalTemperature;
  unsigned short int uiSpectrometerTemperature;
  unsigned char ucInternalRelativeHumidity;
  struct
  {
   unsigned short int uiMean;
   signed short int iStandardDeviation;
  }
  tDarkSpectrum;
  struct
```

```
 {
  float fNitrate;
 }
 tSensor;
 float fAbsorbanceFitResiduals;
 struct
 {
  unsigned short int tuiSpectrum[ N ];
 }
 tOutput;
}
tAPFData;
}
tRPark;
```

45 outputs spectrum - Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Temperature spec.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Relative humidity	BYTE	0.0 to +100% RH	0.5% RH	1
Dark M	SHORTINT	0.0 to +6553.5	0.1	2
Dark SD	SHORTINT	-327.68 to +327.67	0.01	2
Nitrate	FLOAT	IEEE float	IEEE float	4
Absorbance	FLOAT	IEEE float	IEEE float	4
Output 1	SHORTINT	0 to +65535	1	2
...	-	-	-	-
Output 45	SHORTINT	0 to +65535	1	2
				117

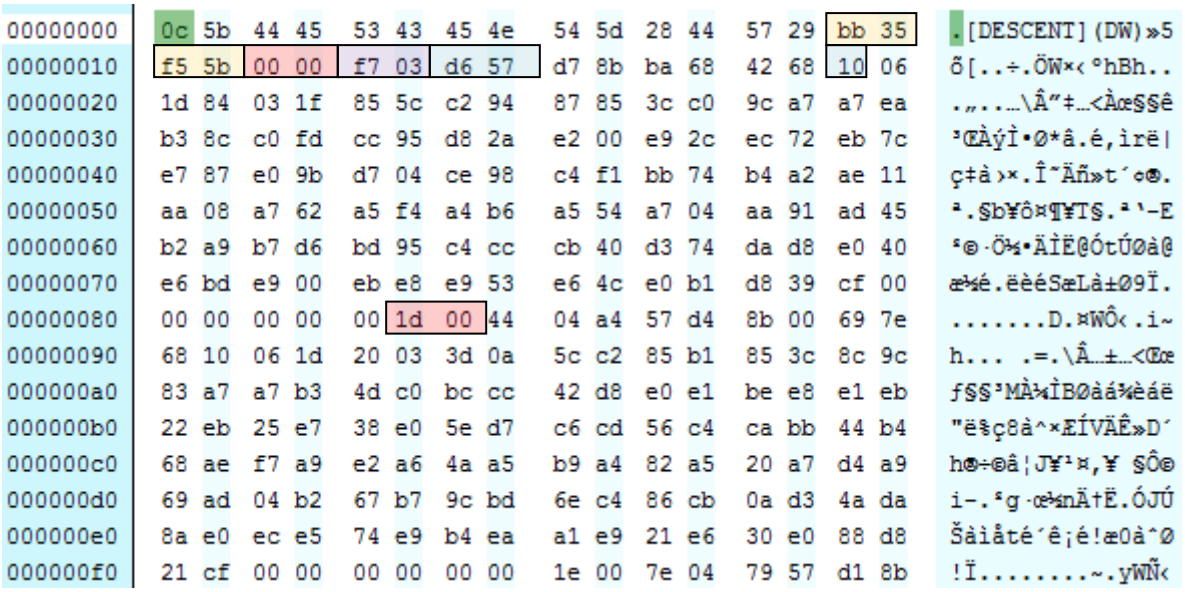
Raw data (navigation) – (RW) or (DW)

```
struct
 {
 unsigned short int uiDateTimeDelta;
 struct
 {
  struct
  {
   unsigned short int uiPressure;
   unsigned short int uiTemperature;
   unsigned short int uiSalinity;
  }
  tCTD;
  unsigned short int uiInternalTemperature;
  unsigned short int uiSpectrometerTemperature;
  unsigned char ucInternalRelativeHumidity;
  struct
  {
   unsigned short int uiMean;
   signed short int iStandardDeviation;
  }
  tDarkSpectrum;
  struct
  {
   float fNitrate;
  }
  tSensor;
  float fAbsorbanceFitResiduals;
  struct
  {
   unsigned short int tuiSpectrum[ N ];
  }
  tOutput;
 }
 tAPFData;
 }
tRNav;
```

45 outputs spectrum - Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Salinity	SHORTINT	0.0 to +50.0 psu	0.001 psu	2
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Temperature spec.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Relative humidity	BYTE	0.0 to +100% RH	0.5% RH	1
Dark M	SHORTINT	0.0 to +6553.5	0.1	2

Dark SD	SHORTINT	-327.68 to +327.67	0.01	2
Nitrate	FLOAT	IEEE float	IEEE float	4
Absorbance	FLOAT	IEEE float	IEEE float	4
Output 1	SHORTINT	0 to +65535	1	2
...	-	-	-	-
Output 45	SHORTINT	0 to +65535	1	2
				115

Example:
[DESCENT]
(DW)
2018-11-21 10:38:51,1.5,17.486,[...],8.0,[...]
2018-11-21 10:39:20,9.2,[...]



Timestamp 1 = 1542796731 (0x5BE435BB) + 0 sec (0x0000) => 2018-11-21 10:38:51
Pressure 1 = 1015 (0x03F7) => ((1015 / 10.0) – 100.0) = 1.5 dbar
Temperature 1 = 22486 (0x57D6) => ((22486 / 1000.0) – 5.0) = 17.486°C
[...]
Humidity 1 = 16 (0x10) => (16 / 2.0) = 8.0 % RH
Timestamp 2 = Timestamp 1 + 29 sec (0x001D) => 2018-11-21 10:39:20
…

6.13. UVP6 sensor (LPMv2)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Number of images** in the record (unsigned char) – code = value
- **Temperature int.** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0
- **Number of particles per size class** (float) – code = value
- **Mean grey level per size class** (unsigned char) – code = value

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucImageNumber;
  unsigned short int uiInternalTemperature;
  float tfObjectCount[ 18 ];
  unsigned char tucGreyLevel[ 18 ];
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Image number	BYTE	1 to +255	1	1
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Particles 1 count	FLOAT	IEEE float	IEEE float	4
...	-	-	-	16*4
Particles 18 count	FLOAT	IEEE float	IEEE float	4
Grey level 1	BYTE	0 to 255	1	1
...	-	-	-	16*1
Grey level 18	BYTE	0 to 255	1	1
				99

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  unsigned char ucImageNumber;
  unsigned short int uiInternalTemperature;
  float tfObjectCount[ 18 ];
  unsigned char tucGreyLevel[ 18 ];
 }
tRNav;
```

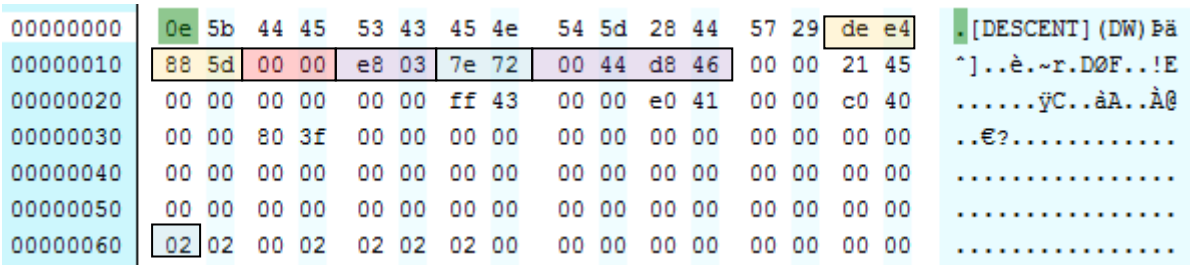
Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Image number	BYTE	1 to +255	1	1
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Particles 1 count	FLOAT	IEEE float	IEEE float	4
...	-	-	-	16*4
Particles 18 count	FLOAT	IEEE float	IEEE float	4
Grey level 1	BYTE	0 to 255	1	1
...	-	-	-	16*1
Grey level 18	BYTE	0 to 255	1	1
				97

Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  unsigned short int uiImageNumber;
  unsigned short int uiInternalTemperature;
  float tfObjectCount[ 18 ];
  unsigned char tucGreyLevel[ 18 ];
 }
tM;
```

Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Image number	SHORTINT	1 to +65535	1	2
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Particles 1 count	FLOAT	IEEE float	IEEE float	4
...	-	-	-	16*4
Particles 18 count	FLOAT	IEEE float	IEEE float	4
Grey level 1	BYTE	0 to 255	1	1
...	-	-	-	16*1
Grey level 18	BYTE	0 to 255	1	1
				98

Example:
[DESCENT]
(DW)
2019-09-23 15:29:34,0,0.00,24.31,27682.0000,2576.0000,0.0000,510.0000,[...],2,2,0,2,2,2,2,0,[...]



Timestamp 1 = 1569252574 (0x5D88E4DE) + 0 sec (0x0000) => 2019-09-23 15:29:34
Pressure 1 = 1015 (0x03E8) => ((1000 / 10.0) – 100.0) = 0.0 dbar
Temperature 1 = 29310 (0x727E) => ((29310 / 1000.0) – 5.0) = 24.310°C
Particle 1/18 1 = 27682.0 (0x46D84400)
[...]
Grey level 1 = 2 (0x02)

6.14. UVP6 sensor (TAXOv2)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Number of images** in the record (unsigned char) – code = value
- **Number of object** in the record (unsigned char) – code = value (1 to 40 max.)
- **Object number** in the record (unsigned char) – code = value (1 to 255)
- **Object index** (unsigned char) – code = value (0 to 39)
- **Mean size per an object** (float) – code = value
- **Mean grey level per an object** (unsigned char) – code = value

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

This data type has the particularity to use a variable size record. Each record could have different size. The size of the packet is determinate by the field “number of object”.

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucImageNumber;
  unsigned char ucObjectNumber;
  struct
   {
    unsigned char ucObjectIndex;
    unsigned char ucObjectNumber;
    float fSize;
    unsigned char ucGreyLevel;
   }
  ttObject[ 0 to 40 ];
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Image number	BYTE	1 to +255	1	1
Object number	BYTE	0 to 40	1	1
Object index 1	BYTE	0 to 39	1	1
Object number 1	BYTE	1 to +255	1	1

Object size 1	FLOAT	IEEE float	IEEE float	4
Object grey level 1	BYTE	0 to +255	1	1
...	-	-	-	-
Object index N	BYTE	0 to 39	1	1
Object number N	BYTE	1 to +255	1	1
Object size N	FLOAT	IEEE float	IEEE float	4
Object grey level N	BYTE	0 to +255	1	1
				8+(N*7)

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  unsigned char ucImageNumber;
  unsigned char ucObjectNumber;
  struct
   {
    unsigned char ucObjectIndex;
    unsigned char ucObjectNumber;
    float fSize;
    unsigned char ucGreyLevel;
   }
  ttObject[ 0 to 40 ];
 }
tRNav;
```

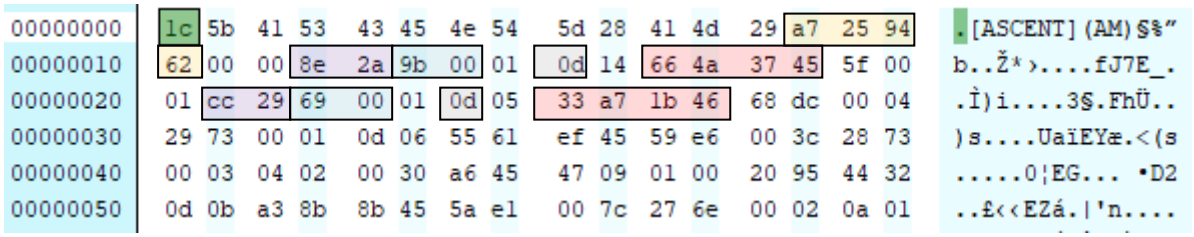
Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Image number	BYTE	1 to +255	1	1
Object number	BYTE	0 to 40	1	1
Object index 1	BYTE	0 to 39	1	1
Object number 1	BYTE	1 to +255	1	1
Object size 1	FLOAT	IEEE float	IEEE float	4
Object grey level 1	BYTE	0 to +255	1	1
...	-	-	-	-
Object index N	BYTE	0 to 39	1	1
Object number N	BYTE	1 to +255	1	1
Object size N	FLOAT	IEEE float	IEEE float	4
Object grey level N	BYTE	0 to +255	1	1
				6+(N*7)

Average data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  unsigned short int uiImageNumber;
  unsigned char ucObjectNumber;
  struct
   {
     unsigned char ucObjectIndex;
     unsigned char ucObjectNumber;
     float fSize;
     unsigned char ucGreyLevel;
   }
  ttObject[ 0 to 40 ];
 }
tM;
```

Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Image number	SHORTINT	1 to +65535	1	2
Object number	BYTE	0 to 40	1	1
Object index 1	BYTE	0 to 39	1	1
Object number 1	BYTE	1 to +255	1	1
Object size 1	FLOAT	IEEE float	IEEE float	4
Object grey level 1	BYTE	0 to +255	1	1
...	-	-	-	-
Object index N	BYTE	0 to 39	1	1
Object number N	BYTE	1 to +255	1	1
Object size N	FLOAT	IEEE float	IEEE float	4
Object grey level N	BYTE	0 to +255	1	1
				7+(N*7)

Example:
[ASCENT]
(AM)
{Record A} 2022-05-30 02:02:15,989.4,155,
{Object index 0 to 12} 0,0.0,0
{Object index 13} 20,2932.65,95
{Object index 14 to 39} 0,0.0,0
{Record B} 2022-05-30 02:06:31,970.0,105
{Object index 0 to 12} 0,0.0,0
{Object index 13} 5,9961.80,104,
{Object index 14 to 39} 0,0.0,0
[...]



Timestamp A = 1653876135 (0x629425A7) => 2022-05-30 02:02:15
Pressure A = 10894 (0x2A8E) => ((10894 / 10.0) – 100.0) = 989.4 dbar
Image number A = 155 (0x009B)
Object number A = 1 (0x01)
Object 1 index A = 13 (0x0D)
Object 1 number A = 20 (0x14)
Object 1 size A = 2932.65 (0x45374A66)
Object 1 grey level A = 95 (0x5F)
Timestamp B = 1653876391 => 1653876135 + 256 (0x0100)
Pressure B = 10700 (0x29CC) => ((10700 / 10.0) – 100.0) =970.0 dbar
Image number B = 105 (0x0069)
Object number B = 1 (0x01)
Object 1 index B = 13 (0x0D)
Object 1 number B = 5 (0x05)
Object 1 size B = 9961.80 (0x461BA733)
Object 1 grey level B = 104 (0x68)
[...]

6.15. UVP6 sensor (BLACKv2)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Number of images** in the record (unsigned char) – code = value
- **Internal temperature** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0
- **Number of particles per size class** (unsigned integer) – code = value

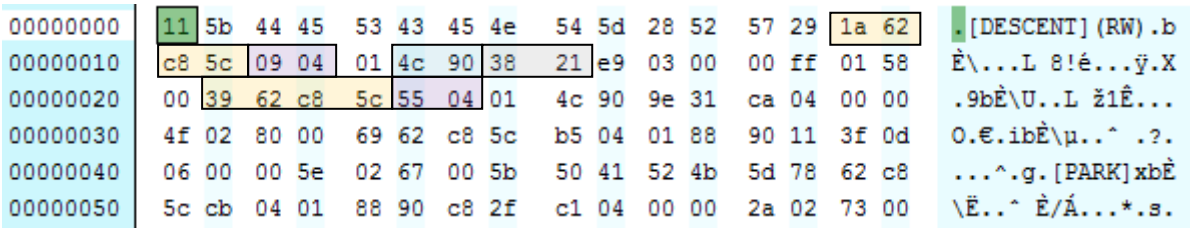
All records are timestamped in absolute value.

Raw data – (RW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucImageNumber;
  unsigned short int uiInternalTemperature;
  unsigned short int tuiObjectCount[ 5 ];
 }
tR;
```

Raw data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Image number	BYTE	1 to +255	1	1
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Particles 1 count	SHORTINT	0 to +65535	1	2
...	-	-	-	3*2
Particles 5 count	SHORTINT	0 to +65535	1	2
				19

Example:
[DESCENT]
(RW)
2019-04-30 14:56:26,3.3,1,31.94,8504,1001,0,511,88
2019-04-30 14:56:57,10.9,1,31.94,12702,1226,0,591,128



Timestamp 1 = 1556636188 (0x5CC8621A) => 2019-04-30 14:56:26
Pressure 1 = 1033 (0x0409) => ((1033 / 10.0) – 100.0) = 3.3 dbar
Temperature 1 = 36940 (0x904C) => ((36940 / 1000.0) – 5.0) = 31.94°C
Object count class_1 1 = 8504 (0x2138)
[...]
Timestamp 2 = 1556636217 (0x5CC86239) => 2019-04-30 14:56:57
Pressure 2 = 1109 (0x0455) => ((1109 / 10.0) – 100.0) = 10.9 dbar
...

6.16. EXTTRIG sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0

All records are timestamped in absolute value. So, there are no processing type identification strings inserted in the data file.

Raw data – (RW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
 }
tR;
```

Raw data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
				6

Example:

[DESCENT]
(RW)
2020-01-20 14:08:53,0.0
2020-01-20 14:08:55,0.0

```
00000000  16 5b 44 45 53 43 45 4e 54 5d 75 b4 25 5e e8 03  .[DESCENT]u´%^è.
00000010  77 b4 25 5e e8 03 7a b4 25 5e e8 03 7d b4 25 5e  w´%^è.z´%^è.}´%^
00000020  e8 03 80 b4 25 5e e8 03 83 b4 25 5e e9 03 86 b4  è.€´%^è.ƒ´%^é.†´
00000030  25 5e ea 03 89 b4 25 5e eb 03 8c b4 25 5e ec 03  %^ê.‰´%^ë.Œ´%^ì.
```

Timestamp 1 = 1579529333 (0x5E25B475) => 2020-01-20 14:08:53
Pressure 1 = 1000 (0x03E8) => ((1000 / 10.0) – 100.0) = 0.0 dbar
Timestamp 2 = 1579529335 (0x5E25B477) => 2020-01-20 14:08:55
Pressure 2 = 1000 (0x03E8) => ((1000 / 10.0) – 100.0) = 0.0 dbar
...

6.17. RAMSESv2/RAMSES2v2 sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Integration time** (unsigned integer) – code = value
- **Depth** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 20.0
- **Inclination** in cdegree (unsigned integer) – code = value * 100.0
- **Timestamp 2** in sec (unsigned char) – code = value * 4.0
- **Dark average** (unsigned integer) – code = value
- **Number of spectrum channels** (unsigned char) – code = value
- **Spectrum channels count** (unsigned integer) – code = value (spectrum size is not constant)

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned short int uiIntegrationTime;
  unsigned short int uiPrePressure;
  unsigned short int uiPostPressure;
  unsigned short int uiPreInclination;
  unsigned short int uiPostInclination;
  unsigned char ucTimestamp2;
  unsigned short int uiDark;
  unsigned char ucSpectrumSize;
  unsigned short int tuiSpectrum[ 245 max ];
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Integration time	SHORTINT	0 to +65535	1	2
Pre-Pressure	SHORTINT	-100 to +2500 dbar	0.05 dbar	2
Post-Pressure	SHORTINT	-100 to +2500 dbar	0.05 dbar	2
Pre-Inclination	SHORTINT	0 to +360°	0.01°	2
Post-Inclination	SHORTINT	0 to +360°	0.01°	2

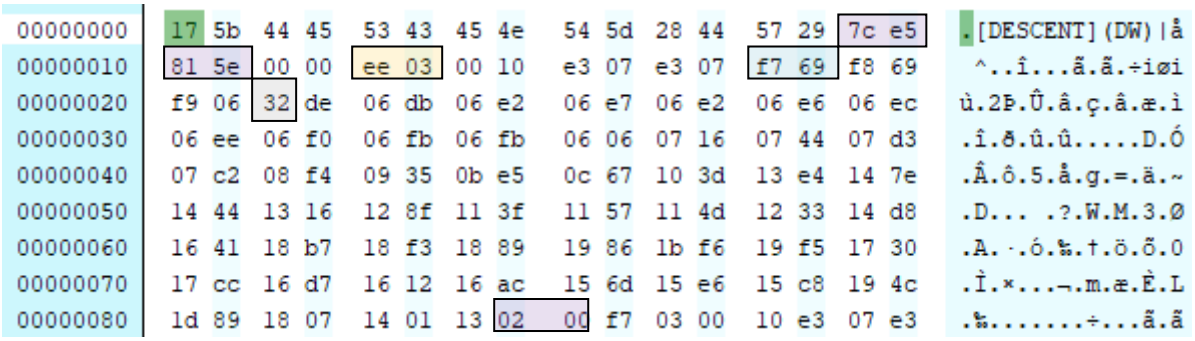
Timestamp 2	BYTE	0.0 to +63.75 sec	0.25 sec	1
Dark average	SHORTINT	0 to +65535	1	2
Channel number N	BYTE	0 to +245	1	1
Channel 1 count	SHORTINT	0 to +65535	1	2
...	-	-	-	(N-2)*2
Channel N count	SHORTINT	0 to +65535	1	2
				20+2N

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  unsigned short int uiIntegrationTime;
  unsigned short int uiPrePressure;
  unsigned short int uiPostPressure;
  unsigned short int uiPreInclination;
  unsigned short int uiPostInclination;
  unsigned char ucTimestamp2;
  unsigned short int uiDark;
  unsigned char ucSpectrumSize;
  unsigned short int tuiSpectrum[ 245 max ];
 }
tRNav;
```

Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Integration time	SHORTINT	0 to +65535	1	2
Pre-Pressure	SHORTINT	-100 to +2500 dbar	0.05 dbar	2
Post-Pressure	SHORTINT	-100 to +2500 dbar	0.05 dbar	2
Pre-Inclination	SHORTINT	0 to +360°	0.01°	2
Post-Inclination	SHORTINT	0 to +360°	0.01°	2
Timestamp 2	BYTE	0.0 to +63.75 sec	0.25 sec	1
Dark average	SHORTINT	0 to +65535	1	2
Channel number N	BYTE	0 to +245	1	1
Channel 1 count	SHORTINT	0 to +65535	1	2
...	-	-	-	(N-2)*2
Channel N count	SHORTINT	0 to +65535	1	2
				18+2N

Example:
[DESCENT]
(DW)
2020-03-30 12:26:36,0.60,4096,0.95,0.95,271.27,271.28,1785,50,1758,1755,1762,[...],4865
2020-03-30 12:26:38,1.50,4096,0.95,0.95,271.27,271.28,1785,50,1758,1755,1762,[...],4865



Timestamp 1 = 1585571196 (0x5E81E57C) => 2020-03-30 12:26:36
Pressure 1 = 1006 (0x03EE) => ((1006 / 10.0) – 100.0) = 0.6 dbar
Pre-Inclination 1 = 27128 (0x69F7) => (27127 * 100.0) = 271.28°
Channel number 1 = 50 (0x32)
[…]
Timestamp 2 = Timestamp 1 + 2 sec (0x0002) => 2020-03-30 12:26:38

/!\ Timestamp 2 is missing in this example (added in v1.02.006 firmware)

6.18. OPUS sensor (LIGHTv2)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Spectrum type** (char) – code = '2' for calibrated spectrum and '4' for raw light spectrum
- **Averaging** (unsigned char) – code = value * 10
- **Flash count** (unsigned char) – code = value * 10
- **Internal temperature** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0
- **Lamp Reference 1** (signed integer) – code = value
- **Number of spectrum channels** (unsigned char) – code = value
- **Spectrum channels count** (unsigned integer) – code = value (spectrum size is not constant)

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  char cSpectrumType;
  unsigned char ucAveraging;
  unsigned char ucFlashCount;
  unsigned short int uiTemperature;
  signed short int iLampReference1;
  unsigned char uiSpectrumSizeN;
  unsigned short int tuiSpectrumN[ 255 max ];
  unsigned char uiSpectrumSizeM;
  unsigned short int tuiSpectrumM[ 255 max ];
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Spectrum type	BYTE	'2': calibrated / '4': raw	1	1
Averaging	BYTE	0 to 25.0	0.1	1
Flash count	BYTE	0 to 25.0	0.1	1
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2

Lamp Reference 1	SHORTINT	-32768 to +32767	1	2
Channel number N	BYTE	0 to +255	1	1
Channel F1 count	SHORTINT	0 to +65535	1	2
...	-	-	-	(N-2)*2
Channel FN count	SHORTINT	0 to +65535	1	2
Channel number M	BYTE	0 to +255	1	1
Channel B1 count	SHORTINT	0 to +65535	1	2
...	-	-	-	(M-2)*2
Channel BM count	SHORTINT	0 to +65535	1	2
				15+2N+2M

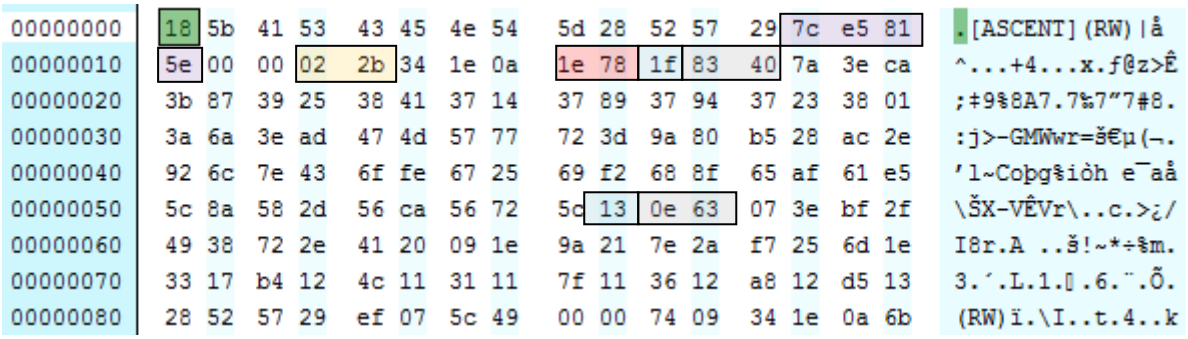
Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  char cSpectrumType;
  unsigned char ucAveraging;
  unsigned char ucFlashCount;
  unsigned short int uiTemperature;
  signed short int iLampReference1;
  unsigned char uiSpectrumSizeN;
  unsigned short int tuiSpectrumN[ 255 max ];
  unsigned char uiSpectrumSizeM;
  unsigned short int tuiSpectrumM[ 255 max ];
 }
tRNav/tM;
```

Raw data encoding (navigation)/ Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Raw record/Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Spectrum type	BYTE	‘2’: calibrated / ‘4’: raw	1	1
Averaging	BYTE	0 to 25.0	0.1	1
Flash count	BYTE	0 to 25.0	0.1	1
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Lamp Reference 1	SHORTINT	-32768 to +32767	1	2
Channel number N	BYTE	0 to +255	1	1
Channel F1 count	SHORTINT	0 to +65535	1	2
...	-	-	-	(N-2)*2
Channel FN count	SHORTINT	0 to +65535	1	2
Channel number M	BYTE	0 to +255	1	1
Channel B1 count	SHORTINT	0 to +65535	1	2
...	-	-	-	(M-2)*2

Channel BM count	SHORTINT	0 to +65535	1	2
				13+2N+2M

Example:
[ASCENT]
(RW)
2020-03-30 12:26:36,1001.0,4,3.0,1.0,25.750,31,16515,[…],19,25358,[…]



Timestamp 1 = 1585571196 (0x5E81E57C) => 2020-03-30 12:26:36
Pressure 1 = 11010 (0x2B02) => ((11010 / 10.0) – 100.0) = 1001.0 dbar
Spectrum type 1 = “4” (0x34)
Averaging 1 = 30 (0x1E) => (30 / 10.0) = 3.0
Flash count 1 = 10 (0x0A) => (10 / 10.0) = 1.0
Temperature 1 = 30730 (0x781E) => ((30750 / 1000.0) – 5.0) = 25.750°C
Spectrum size N 1 = 31 (0x1F)
Channel F1 1 = 16515 (0x4083)
[...]
Spectrum size M 1 = 19 (0x13)
Channel B1 1 = 25358 (0x630E)
[…]

/!\ Led Reference 1 is missing in this example (added in v1.02.006 firmware)

6.19. OPUS sensor (BLACKv2)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Averaging** (unsigned char) – code = value * 10
- **Flash count** (unsigned char) – code = value * 10
- **Internal temperature** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0
- **Lamp Reference 1** (signed integer) – code = value
- **Dark mean** (float) – code = value
- **Dark standard deviation** (float) – code = value

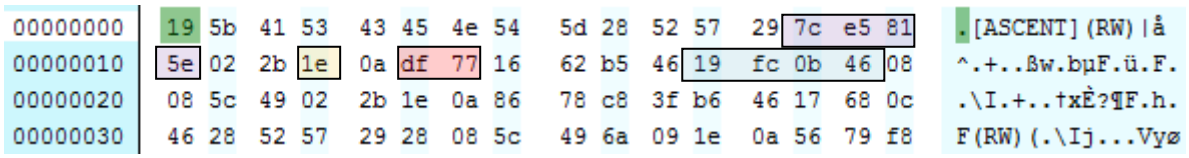
All records are timestamped in absolute value.

Raw data – (RW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucAveraging;
  unsigned char ucFlashCount;
  unsigned short int uiTemperature;
  signed short int iLampReference1;
  float fDarkMean;
  float fDarkStdDev;
 }
tR;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Averaging	BYTE	0 to 25.0	0.1	1
Flash count	BYTE	0 to 25.0	0.1	1
Temperature int.	SHORTINT	-5.0 to +40.0°C	0.001°C	2
Lamp Reference 1	SHORTINT	-32768 to +32767	1	2
Dark mean	FLOAT	IEEE float	IEEE float	4
Dark standard dev	FLOAT	IEEE float	IEEE float	4
				20

Example:
[ASCENT]
(RW)
2020-03-30 12:26:36,1001.0,3.0,1.0,25.687,23217.0,8959.0



Timestamp 1 = 1585571196 (0x5E81E57C) => 2020-03-30 12:26:36
Pressure 1 = 11010 (0x2B02) => ((11010 / 10.0) – 100.0) = 1001.0 dbar
Averaging 1 = 30 (0x1E) => (30 / 10.0) = 3.0
Flash count 1 = 10 (0x0A) => (10 / 10.0) = 1.0
Temperature 1 = 30687 (0x77DF) => ((30687 / 1000.0) – 5.0) = 25.687°C
Dark mean 1 = 23217.0 (0x46B56216)
Dark std deviation 1 = 8959.0 (0x460BFC19)
[…]

/!\ Led Reference 1 is missing in this example (added in v1.02.006 firmware)

6.20. MPE sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Voltage** in xV (float) – code = value
- **Temperature** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  float fVoltage;
  unsigned short int uiTemperature;
 }
 tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	FLOAT	-5.0e-03 to +1.6e+05 xV	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
				12

Raw data (navigation) – (RW) or (DW)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  float fVoltage;
  unsigned short int uiTemperature;
 }
 tRNav;
```

Raw data encoding (navigation)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	FLOAT	-5.0e-03 to +1.6e+05 xV	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
				10

Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiAveragePressure;
  float fAverageVoltage;
  unsigned short int uiAverageTemperature;
 }
 tM;
```

Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535 sec	1 sec	2
Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Voltage	FLOAT	-5.0e-03 to +1.6e+05 xV	IEEE float	4
Temperature	SHORTINT	-5.0 to +40.0°C	0.001°C	2
				10

Example:
[DESCENT]
(DW)
2021-01-28 15:22:56,0.00,4.567e-04,15.3

```
00000000  1a 5b 44 45  53 43 45 4e  54 5d 28 44  57 29 d0 d6  .[DESCENT](DW)ÐÖ
00000010  12 60 00 00  e8 03 3d 71  ef 39 4c 4f  28 44 57 29  .`..è.=qï9LO(DW)
00000020  de d6 12 60  00 00 ec 03  3d 71 ef 39  4c 4f 28 44  ÞÖ.`..ì.=qï9LO(D
```

Timestamp 1 = 1611847376 (0x6012D6D0) => 2021-01-28 15:22:56
Pressure 1 = 1000 (0x03E8) => ((1000 / 10.0) – 100.0) = 0.0 dbar
Voltage 1 = 4.567e-04 (0x39EF713D)
Temperature 1 = 20300 (0x4F4C) => ((20300 / 1000.0) – 5.0) = 15.3°C

6.21. HYDROC sensor (M or C)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Acquisition mode**: 0 = warm-up, 1 = zero, 2 = flush and 3 = measure (byte) – code = value
- **Raw signal** (unsigned integer) – code = value
- **Pressure** (internal) in mbar and -750 mbar offset (unsigned integer) – code = (value - 750.0) * 100.0
- **Temperature** in m°C and +5.0°C offset (unsigned integer) – code = (value + 5.0) * 1000.0
- **Humidity** in % (unsigned integer) – code = value * 200.0
- **Pump power** in W (unsigned char) – code = value * 100.0
- **Supply voltage** in V (unsigned char) – code = value * 10.0
- **Total current** in mA (unsigned integer) – code = value * 10.0
- **Runtime** in sec (unsigned long integer) – code = value

During the "ascent" navigation phase, the first record of each processing is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucAcquisitionMode;
  unsigned short int uiRawDetector;
  unsigned short int uiRawReference;
  unsigned short int uiPressurIn;
  unsigned short int uiPressureNDIR;
  unsigned short int uiTemperatureNDIR;
  unsigned short int uiTemperatureGas;
  unsigned short int uiHumidityGas;
  unsigned char ucPumpPower;
  unsigned char ucSupplyVoltage;
  unsigned short int uiTotalCurrent;
  unsigned long int uliRuntime;
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Acquisition mode	BYTE	0 to 3	1	1
Raw detector	SHORTINT	0 to +65535	1	2
Raw reference	SHORTINT	0 to +65535	1	2
Pressure In	SHORTINT	+850.0 to +1150.0 mbar	0.01 mbar	2
Pressure NDIR	SHORTINT	+850.0 to +1150.0 mbar	0.01 mbar	2
Temperature NDIR	SHORTINT	-5.0 to +60.0°C	0.001°C	2
Temperature Gas	SHORTINT	-5.0 to +60.0°C	0.001°C	2
Humidity Gas	SHORTINT	0.0 to +100.0 %	0.005 %	2
Pump Power	BYTE	0.0 to +2.0 W	0.01 W	1
Supply Voltage	BYTE	0.0 to +16.0 V	0.1 V	1
Total Current	SHORTINT	0.0 to +2.0 A	0.1 mA	2
Runtime	LONGINT	0 to +4294967295 sec	1 sec	4
				29

Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  unsigned char ucAcquisitionMode;
  unsigned short int uiRawDetector;
  unsigned short int uiRawReference;
  unsigned short int uiPressurIn;
  unsigned short int uiPressureNDIR;
  unsigned short int uiTemperatureNDIR;
  unsigned short int uiTemperatureGas;
  unsigned short int uiHumidityGas;
  unsigned char ucPumpPower;
  unsigned char ucSupplyVoltage;
  unsigned short int uiTotalCurrent;
  unsigned long int uliRuntime;
 }
tRNav/tM;
```

Raw data encoding (navigation)/ Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Raw record/Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Acquisition mode	BYTE	0 to 3	1	1
Raw detector	SHORTINT	0 to +65535	1	2
Raw reference	SHORTINT	0 to +65535	1	2
Pressure In	SHORTINT	+850.0 to +1150.0 mbar	0.01 mbar	2
Pressure NDIR	SHORTINT	+850.0 to +1150.0 mbar	0.01 mbar	2
Temperature NDIR	SHORTINT	-5.0 to +60.0°C	0.001°C	2
Temperature Gas	SHORTINT	-5.0 to +60.0°C	0.001°C	2
Humidity Gas	SHORTINT	0.0 to +100.0 %	0.005 %	2
Pump Power	BYTE	0.0 to +2.0 W	0.01 W	1
Supply Voltage	BYTE	0.0 to +16.0 V	0.1 V	1
Total Current	SHORTINT	0.0 to +2.0 A	0.1 mA	2
Runtime	LONGINT	0 to +4294967295 sec	1 sec	4
				27

Example:
[ASCENT]
(RW)
2021-11-08
13:57:05,142.00,3,12094,18595,1034.84,1046.18,44.200,28.783,28.780,0.4,11.4,425.1,250136

```
00000000  1e 5b 41 53 43 45 4e 54  5d 28 52 57 29 b1 2c 89  .[ASCENT](RW)±,‰
00000010  61 00 00 74 09 03 3e 2f  a3 48 44 6f b2 73 30 c0  a..t..>/£HDo²s0À
00000020  f7 83 7c 16 04 72 9b 10  18 d1 03 00 01 00 74 09  ÷ƒ|..r›..Ñ....t.
00000030  03 3a 2f a3 48 3f 6f bf  73 30 c0 8d 83 67 16 04  .:/£H?o¿s0À ƒg..
00000040  72 27 0e 19 d1 03 00 00  00 74 09 03 3a 2f a3 48  r'..Ñ....t..:/£H
00000050  3f 6f bf 73 30 c0 8d 83  67 16 04 72 27 0e 19 d1  ?o¿s0À ƒg..r'..Ñ
00000060  03 00 01 00 74 09 03 37  2f a4 48 38 6f a6 73 30  ....t..7/¤H8o¦s0
```

Timestamp = 1636379825 (0x89612CB1) => 2021-11-08 13:57:05
Pressure = 2420 (0x0974) => ((2420 / 10.0) – 100.0) = 142.0 dbar
Acquisition mode = 03 (0x03)
[...]
Pressure In = 28484 (0x6F44) => ((28484 / 100) + 750.0) = 1034.84 mbar
[…]
Total current = 4251 (0x109B) => (4251 / 10.0) = 425.1 mA

6.22. IMU sensor (R)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Temperature** (internal) raw value (signed integer) – code = value
- **Acceleration** raw value (signed integer) – code = value
- **Angular rate** raw value (signed integer) – code = value
- **Magnetic field** raw value (signed integer) – code = value

During the "ascent" navigation phase, the first record of each processing is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  signed short int iTemperature;
  signed short int iAccelerometerX;
  signed short int iAccelerometerY;
  signed short int iAccelerometerZ;
  signed short int iGyroscopeX;
  signed short int iGyroscopeY;
  signed short int iGyroscopeZ;
  signed short int iMagnetometerX;
  signed short int iMagnetometerY;
  signed short int iMagnetometerZ;
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-32768 to +32767	1	2
Accelerometer X	SHORTINT	-32768 to +32767	1	2
Accelerometer Y	SHORTINT	-32768 to +32767	1	2
Accelerometer Z	SHORTINT	-32768 to +32767	1	2
Gyroscope X	SHORTINT	-32768 to +32767	1	2
Gyroscope Y	SHORTINT	-32768 to +32767	1	2
Gyroscope Z	SHORTINT	-32768 to +32767	1	2
Magnetometer X	SHORTINT	-32768 to +32767	1	2
Magnetometer Y	SHORTINT	-32768 to +32767	1	2

Magnetometer Z	SHORTINT	-32768 to +32767	1	2
				26

Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  signed short int iTemperature;
  signed short int iAccelerometerX;
  signed short int iAccelerometerY;
  signed short int iAccelerometerZ;
  signed short int iGyroscopeX;
  signed short int iGyroscopeY;
  signed short int iGyroscopeZ;
  signed short int iMagnetometerX;
  signed short int iMagnetometerY;
  signed short int iMagnetometerZ;
 }
 tRNav/tM;
```

Raw data encoding (navigation)/ Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Raw record/Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	-32768 to +32767	1	2
Accelerometer X	SHORTINT	-32768 to +32767	1	2
Accelerometer Y	SHORTINT	-32768 to +32767	1	2
Accelerometer Z	SHORTINT	-32768 to +32767	1	2
Gyroscope X	SHORTINT	-32768 to +32767	1	2
Gyroscope Y	SHORTINT	-32768 to +32767	1	2
Gyroscope Z	SHORTINT	-32768 to +32767	1	2
Magnetometer X	SHORTINT	-32768 to +32767	1	2
Magnetometer Y	SHORTINT	-32768 to +32767	1	2
Magnetometer Z	SHORTINT	-32768 to +32767	1	2
				24

Example:
[ASCENT]
(RW)
2022-06-14 15:37:25,17.50,-3744,-376,16203,-937,14,6,232,3354,-4286,556

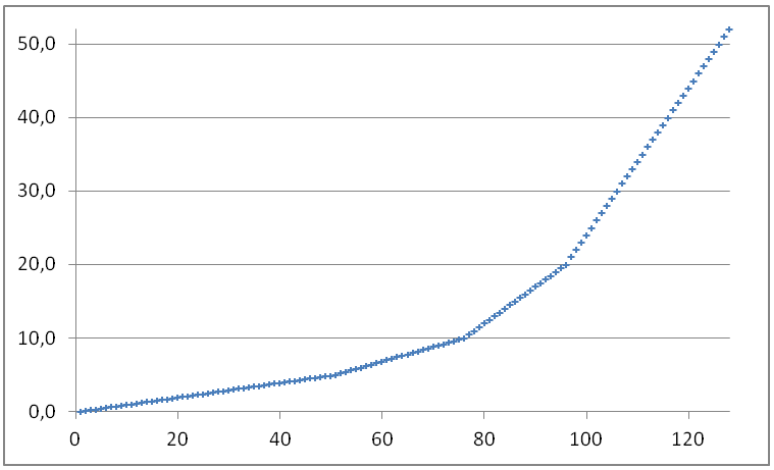
```
00000000  21 5b 41 53 43 45 4e 54  5d 28 52 57 29 35 ab a8  ![ASCENT](RW)5«¨
00000010  62 00 00 97 04 60 f1 88  fe 4b 3f 57 fc 0e 00 06  b..—.`ñˆþK?Wü...
00000020  00 e8 00 1a 0d 42 ef 2c  02 01 00 97 04 60 f1 88  .è...Bï,...—.`ñˆ
00000030  fe 48 3f 6b fc 14 00 00  00 d9 00 76 0d 15 ef e9  þH?kü....Ù.v..ïé
00000040  01 02 00 97 04 30 f1 89  fe 49 3f 86 fc 18 00 fc  ...—.0ñ‰þI?†ü..ü
```

Timestamp = 1655221045 (0x62A8AB35) => 2022-06-16 14 15:37:25
Pressure = 1175 (0x0497) => ((1175 / 10.0) – 100.0) = 17.5 dbar
Accelerometer X = - 376 (0xFE88)
[…]

6.23. IMU sensor (TH)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Tilt** angle in degrees (unsigned char) is coded using a changing resolution mechanism (128 coding values). The aim is to ensure high resolution when tilt angle value is small while keeping a wide coding range [0.0 to +52.0].
  Tilt_code = code & 0x7F
  Value in range [0.0 to 5.0], 0.1 degree resolution: Tilt_code = ((value – 0)* 10) + 0
  Value in range [5.2 to 10.0], 0.2 degree resolution: Tilt_code = ((value – 5) * 5) + 50
  Value in range [10.5 to 20.0], 0.5 degree resolution: Tilt_code = ((value – 10) * 2) + 75
  Value in range [21.0 to 52.0], 1.0 degree resolution: Tilt_code = ((value – 20) * 1) + 95



- **Heading sign** is coded in the 8th bit of the Tilt: if heading is negative => code & 0x80 = 0x80
- **Heading** angle in degrees (unsigned char) – code = value

During the "ascent" navigation phase, the first record of each processing is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

Raw data (park) – (RW) or (DW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucTilt;
  unsigned char ucHeading;
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Tilt	BYTE	0.0 to +52.0 deg	changing	1
Heading	BYTE	-180.0 to +180.0 deg (*)	1.0 deg	1
				8

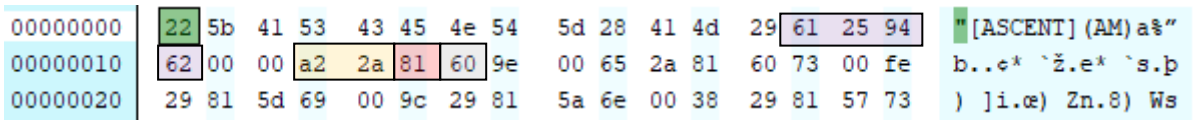
Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
 {
  unsigned short int uiDateTimeDelta;
  unsigned short int uiPressure;
  unsigned char ucTilt;
  unsigned char ucHeading;
 }
tRNav/tM;
```

Raw data encoding (navigation)/ Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	0 to +65535	1 sec	2
Raw record/Average record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Tilt	BYTE	0.0 to +52.0 deg (*)	changing	1
Heading	BYTE	-180.0 to +180.0 deg (*)	1.0 deg	1
				6

(*) Heading sign is coded in the tilt angle 8th bit.

Example:
[ASCENT]
(AM)
2022-05-30 02:01:05,991.40,0.1,-96.0



Timestamp	= 1653876065 (0x62942561) => 2022-05-30 14 02:01:05
Pressure	= 10994 (0x2AA2) => ((10994 / 10.0) – 100.0) =991.4 dbar
Heading sign	= test negative flag (0x81 & 0x80 mask) = 0x80 => negative
Tilt	= 1 (0x01 = 0x81 & 0x7F mask) => ((1 – 0) /10) – 0 = 0.1°
Heading	= 96 (0x60) => - 96° (with negative flag)

6.24. WAVE sensor (R)

Data is encoded as follows:

- **Timestamping** of the record
- **Acceleration** raw value (signed integer) – code = value
- **Magnetic field** raw value (signed integer) – code = value

During the "surface" navigation phase, a burst record of N points is achieved at 4 Hz.

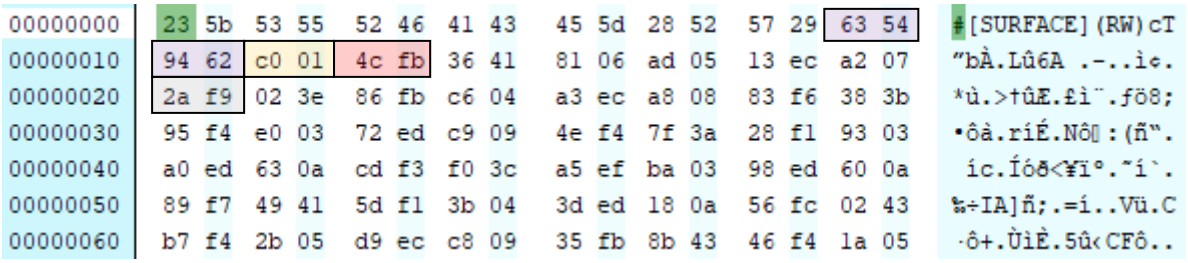
Raw data (surface) – (RW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiN;
  signed short int iAccelerometerX1;
  signed short int iAccelerometerY1;
  signed short int iAccelerometerZ1;
  signed short int iMagnetometerX1;
  signed short int iMagnetometerY1;
  signed short int iMagnetometerZ1;
  […]
  signed short int iAccelerometerXN;
  signed short int iAccelerometerYN;
  signed short int iAccelerometerZN;
  signed short int iMagnetometerXN;
  signed short int iMagnetometerYN;
  signed short int iMagnetometerZN;
 }
tRSurface;
```

Raw data encoding (surface)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
N	SHORTINT	1 to 512	1	2
Accelerometer X1	SHORTINT	-32768 to +32767	1	2
Accelerometer Y1	SHORTINT	-32768 to +32767	1	2
Accelerometer Z1	SHORTINT	-32768 to +32767	1	2
Magnetometer X1	SHORTINT	-32768 to +32767	1	2
Magnetometer Y1	SHORTINT	-32768 to +32767	1	2
Magnetometer Z1	SHORTINT	-32768 to +32767	1	2
…	…	…	…	…
Accelerometer XN	SHORTINT	-32768 to +32767	1	2
Accelerometer YN	SHORTINT	-32768 to +32767	1	2
Accelerometer ZN	SHORTINT	-32768 to +32767	1	2
Magnetometer XN	SHORTINT	-32768 to +32767	1	2
Magnetometer YN	SHORTINT	-32768 to +32767	1	2

Magnetometer ZN	SHORTINT	-32768 to +32767	1	2
				6+(N*12)

Example:
[SURFACE]
(RW)
2022-05-30 05:21:39,448,-1204,16694,1665,1453,-5101,1954,-1750,[…]



Timestamp = 1653888099 (0x62945463) => 2022-05-30 14 05:21:39
N = 448 (0x01C0)
Accelerometer X1 = -1204 (0xFB4C)
[…]
Accelerometer X2 = -1750 (0xF92A)
[…]

6.25. FLTRIDER sensor

Use Rockland Scientific decoder to process data.

6.26. PAL sensor (R+W)

Data is encoded as follows:

- **Timestamping** of the record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Rain** estimation in mm/h (unsigned char) – code = (value * 2.0)
- **Wind** estimation in m/s (unsigned char) – code = (value * 5.0)
- **Stationarity** criterion (unsigned char) – under evaluation
- **Classification** criterion (unsigned char) – under evaluation

During the "drift" phases, all records are timestamped in absolute value.

Raw data (park) – (RW)

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucRain;
  unsigned char ucWind;
  unsigned char ucStationarity;
  unsigned char ucClassification;
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Rain	BYTE	0.0 to +127.5 mm/h	0.5 mm/h	1
Wind	BYTE	0.0 to 51.0 m/s (183.6 km/h)	0.2 m/s	1
Stationarity	BYTE	-	-	1
Classification	BYTE	-	-	1
				10

6.27. PAL sensor (HR)

Data is encoded as follows:

- **Timestamping** of the record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Stationarity** criterion (unsigned char) – under evaluation
- **Sound level offset** in dB (unsigned char) – code = value * 5.0
- **Sound levels** in dB (unsigned char) – code = (value * 5.0) - offset

During the "drift" phases, all records are timestamped in absolute value.

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucStationarity;
  unsigned char ucOffset;
  unsigned char tucAmplitude[ 9 ];
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Stationarity	BYTE	-	-	1
Offset	BYTE	0.0 to +50.0 dB	0.2 dB	1
63 Hz	BYTE	0.0 to +51.0 dB	0.2 dB	1
125 Hz	BYTE	0.0 to +51.0 dB	0.2 dB	1
400 Hz	BYTE	0.0 to +51.0 dB	0.2 dB	1
1 kHz	BYTE	0.0 to +51.0 dB	0.2 dB	1
2 kHz	BYTE	0.0 to +51.0 dB	0.2 dB	1
5 kHz	BYTE	0.0 to +51.0 dB	0.2 dB	1
8 kHz	BYTE	0.0 to +51.0 dB	0.2 dB	1
12.5 kHz	BYTE	0.0 to +51.0 dB	0.2 dB	1
20 kHz	BYTE	0.0 to +51.0 dB	0.2 dB	1
				17

6.28. PAL sensor (HD)

Data is encoded as follows:

- **Timestamping** of the record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Stationarity** criterion (unsigned char) – under evaluation
- **Sound level offset** in dB (unsigned char) – code = value * 5.0
- **Sound levels** in dB (unsigned char) – code = (value * 2.0) - offset

During the "drift" phases, all records are timestamped in absolute value.

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucStationarity;
  unsigned char ucOffset;
  unsigned char tucAmplitude[ 9 ];
 }
tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Stationarity	BYTE	-	-	1
Offset	BYTE	0.0 to +50.0 dB	0.2 dB	1
63 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
125 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
400 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
1 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
2 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
5 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
8 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
12.5 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
20 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
				17

6.29. PAL sensor (F)

Data is encoded as follows:

- **Timestamping** of the record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Sound level offset** in dB (unsigned char) – code = value * 5.0
- **Sound levels** in dB (unsigned char) – code = (value * 2.0) - offset

During the "drift" phases, all records are timestamped in absolute value.

```
struct
 {
  unsigned long int uliDateTime;
  unsigned short int uiPressure;
  unsigned char ucStationarity;
  unsigned char ucOffset;
  unsigned char tucAmplitude[ 23 ];
 }
 tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Stationary	BYTE	-	-	1
Offset	BYTE	0.0 to +50.0 dB	0.2 dB	1
63 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
100 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
125 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
160 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
400 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
500 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
630 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
800 Hz	BYTE	0.0 to +127.0 dB	0.5 dB	1
1 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
1.25 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
1.6 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
2 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
2.5 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
3.15 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
4 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
5 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
6.3 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
8 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1

10 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
12.5 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
16 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
20 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
25 kHz	BYTE	0.0 to +127.0 dB	0.5 dB	1
				30

6.30. TRIDENTE sensor (3 channels)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Channel value** (float) – code = value

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW)

```
struct
{
 unsigned long int uliDateTime;
 unsigned short int uiPressure;
 float fChannel1;
[...]
 float fChannel3;
}
 tRPark;
```

3 channels - Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1	FLOAT	-	IEEE float	4
Channel 2	FLOAT	-	IEEE float	4
Channel 3	FLOAT	-	IEEE float	4
				18

Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiPressure;
 float fChannel1;
[...]
 float fChannel3;
}
 tRNav;
```

3 channels - Raw data encoding (navigation) / Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	-	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1	FLOAT	-	IEEE float	4
Channel 2	FLOAT	-	IEEE float	4
Channel 3	FLOAT	-	IEEE float	4
				16

6.31. TRIDENTE sensor (9 channels)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Channel value** (float) – code = value

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW)

```
struct
{
 unsigned long int uliDateTime;
 unsigned short int uiPressure;
 float fChannel1;
[...]
 float fChannel9;
}
 tRPark;
```

3 channels - Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1	FLOAT	-	IEEE float	4
Channel 2	FLOAT	-	IEEE float	4
Channel 3	FLOAT	-	IEEE float	4
Channel 4	FLOAT	-	IEEE float	4
Channel 5	FLOAT	-	IEEE float	4
Channel 6	FLOAT	-	IEEE float	4
Channel 7	FLOAT	-	IEEE float	4
Channel 8	FLOAT	-	IEEE float	4
Channel 9	FLOAT	-	IEEE float	4
				42

Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiPressure;
 float fChannel1;
[...]
 float fChannel9;
}
 tRNav;
```

3 channels - Raw data encoding (navigation) / Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	-	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1	FLOAT	-	IEEE float	4
Channel 2	FLOAT	-	IEEE float	4
Channel 3	FLOAT	-	IEEE float	4
Channel 4	FLOAT	-	IEEE float	4
Channel 5	FLOAT	-	IEEE float	4
Channel 6	FLOAT	-	IEEE float	4
Channel 7	FLOAT	-	IEEE float	4
Channel 8	FLOAT	-	IEEE float	4
Channel 9	FLOAT	-	IEEE float	4
				40

6.32. SENSO sensor (2 channels)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Channel count** (unsigned integer) – code = value

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW)

```
struct
{
 unsigned long int uliDateTime;
 unsigned short int uiPressure;
 unsigned short uiChannel1;
 unsigned short uiChannel2;
}
 tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65535	1	2
Channel 2 count	SHORTINT	0 to +65535	1	2
				10

Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiPressure;
unsigned short uiChannel1;
 unsigned short uiChannel2;
}
 tRNav;
```

Raw data encoding (navigation)/ Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	-	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65535	1	2
Channel 2 count	SHORTINT	0 to +65535	1	2
				8

6.33. SENSO sensor (3 channels)

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Channel count** (unsigned integer) – code = value

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW)

```
struct
{
 unsigned long int uliDateTime;
 unsigned short int uiPressure;
 unsigned short uiChannel1;
 unsigned short uiChannel2;
 unsigned short uiChannel3;
}
 tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65535	1	2
Channel 2 count	SHORTINT	0 to +65535	1	2
Channel 3 count	SHORTINT	0 to +65535	1	2
				12

Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiPressure;
unsigned short uiChannel1;
 unsigned short uiChannel2;
 unsigned short uiChannel3;
}
 tRNav;
```

Raw data encoding (navigation)/ Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	-	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Channel 1 count	SHORTINT	0 to +65535	1	2
Channel 2 count	SHORTINT	0 to +65535	1	2
Channel 3 count	SHORTINT	0 to +65535	1	2
				10

6.34. RINKO sensor

Data is encoded as follows:

- **Timestamping** of each record
- **Pressure** in cbar and +100 dbar offset (unsigned integer) – code = (value + 100.0) * 10.0
- **Temperature count** (unsigned integer) – code = value
- **Oxygen count** (unsigned integer) – code = value
- **LED accumulated time** (unsigned long integer) – code = value

During the "descent" and "ascent" navigation phases, the first record of each processing area is timestamped in absolute value (reference date), then the recordings for the next records are timestamped in relative value by encoding the date in relation to the reference date (deviation in seconds).

During the "drift" phases, all records are timestamped in absolute value.

The available recording structures are as follows:

Raw data (park) – (RW)

```
struct
{
 unsigned long int uliDateTime;
 unsigned short int uiPressure;
 unsigned short uiTemperature;
 unsigned short uiOxygen;
 unsigned long uliLED;
}
 tRPark;
```

Raw data encoding (park)				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	EPOCH	-	1 sec	4
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	0 to +65535	1	2
Oxygen	SHORTINT	0 to +65535	1	2
LED	LONGINT	0 to +4294967295	1	4
				14

Raw data (navigation) – (RW) or (DW)/Averaged data – (AM)

```
struct
{
 unsigned short int uiDateTimeDelta;
 unsigned short int uiPressure;
unsigned short uiTemperature;
 unsigned short uiOxygen;
 unsigned long uliLED;
}
 tRNav;
```

Raw data encoding (navigation)/ Average data encoding				
Field	**Format**	**Range**	**Resolution**	**Size (Byte)**
Timestamp	SHORTINT	-	1 sec	2
Raw record				
Pressure	SHORTINT	-100 to +2500 dbar	0.1 dbar	2
Temperature	SHORTINT	0 to +65535	1	2
Oxygen	SHORTINT	0 to +65535	1	2
LED	LONGINT	0 to +4294967295	1	4
				12

7. Annexes

7.1. Annex A – Argo metadata table

The table below contains the information exported by the float in its metadata file and the associated keys in the Argo database.

NKE	KEY	VALUE
PLATFORM_INFORMATION		
-	PLATFORM_FAMILY	FLOAT
-	PLATFORM_MAKER	NKE
-	FLOAT_SAIL_ID	*Not available*
-	WMO_NUMBER	*Not available*
-	STANDARD_FORMAT_ID	*Not available*
-	MANUAL_VERSION	*Not available*
-	CORIOLIS_DECODER_VERSION	*Not available*
PROFILER		
SN	FLOAT_SERIAL_NUMBER	*SN*
ID	*Not used*	-
Model	PLATFORM_TYPE	*Model*
-	WMO_INST_TYPE	834
-	BLUETOOTH_NUMBER	*Deducted from ID*
TELECOM		
Type	*Not used*	-
IMEI	IMEI_NUMBER	*IMEI*
CID	SIM_NUMBER	*CID*
Login	LOGIN_NAME	*Login*
BEHAVIOUR		
Options	*Not used*	-
Energy	*Not used*	-
HARDWARE		
HARDWARE.CONTROL_BOARD		
Model	CONTROLLER_BOARD_TYPE_PRIMARY	*Model*
SN	CONTROLLER_BOARD_SERIAL_NO_PRIMARY	*SN*
Firmware	FIRMWARE_VERSION	*Firmware*
Checksum	FIRMWARE_CHECKSUM	*Checksum*
SDCard	*Not used*	-
HARDWARE.MEASURE_BOARD		
Model	CONTROLLER_BOARD_TYPE_SECONDARY	*Model*
SN	CONTROLLER_BOARD_SERIAL_NO_SECONDARY	*SN*
Firmware	*Not used*	-
Checksum	*Not used*	-
HARDWARE.EXTENSION_BOARD		

Model	*Not used*	-
HARDWARE.HULL		
Model	*Not used*	-
HARDWARE.BATTERY		
PACK_N	-	N (1 or 2)
Type	BATTERY_TYPE	*Type*
Capacity	BATTERY_PACKS	*N*Capacity*
Voltage	*Not used*	-
SENSOR_INFORMATION		
SENSORS		
SENSOR_SBE41		
SENSOR_SBE41.SENSOR		
SN	-	-
Model	*To convert*	*SBE41-CP*
SENSOR_SBE41.SBE41_BOARD		
Firmware	*Not used*	-
SENSOR_SBE41.TEMPERATURE_COEFF		
ta0, ta1, ta2, ta3	*Not used*	-
	SENSOR	CTD_TEMP
	SENSOR_MAKER	SBE
	SENSOR_MODEL	SBE41CP
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_SBE41.CONDUCTIVITY_COEFF		
g, h, i, j, ctcor, ...	*Not used*	-
	SENSOR	CTD_CNDC
	SENSOR_MAKER	SBE
	SENSOR_MODEL	SBE41CP
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_SBE41.SENSOR_PRESSURE		
SN	*Not used*	-
	SENSOR	CTD_PRES
	SENSOR_MAKER	DRUCK
	SENSOR_MODEL	SBE41CP
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_DO		
SENSOR_DO.SENSOR		
SN	-	-
Model	*To convert*	*DO4330*
	SENSOR	OPTODE_DOXY
	SENSOR_MAKER	AANDERAA
	SENSOR_MODEL	AANDERAA_OPTODE_4330
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_DO.PHASE_COEFF		

c0	*Not used*	-
SENSOR_DO.SVU_FOIL_COEFF		
c0, c1, c2, ..., c6	*Not used*	-
SENSOR_OCR		
SENSOR_OCR.SENSOR		
SN	-	-
Model	*To convert*	*OCR504*
SENSOR_OCR.CHANNEL_01		
a0, a1, im	*Not used*	-
	SENSOR	RADIOMETER_DOWN_IRR380
	SENSOR_MAKER	SATLANTIC
	SENSOR_MODEL	SATLANTIC_OCR504_ICSW
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_OCR.CHANNEL_02		
a0, a1, im	*Not used*	-
	SENSOR	RADIOMETER_DOWN_IRR412
	SENSOR_MAKER	SATLANTIC
	SENSOR_MODEL	SATLANTIC_OCR504_ICSW
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_OCR.CHANNEL_03		
a0, a1, im	*Not used*	-
	SENSOR	RADIOMETER_DOWN_IRR490
	SENSOR_MAKER	SATLANTIC
	SENSOR_MODEL	SATLANTIC_OCR504_ICSW
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_OCR.CHANNEL_04		
a0, a1, im	*Not used*	-
	SENSOR	RADIOMETER_PAR
	SENSOR_MAKER	SATLANTIC
	SENSOR_MODEL	SATLANTIC_OCR504_ICSW
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_ECO		
SENSOR_ECO.SENSOR		
SN	-	-
Model	*To convert*	*ECO1, ECO2 or ECO3*
Type	*Not used*	-
SENSOR_ECO.CHANNEL_01		
sf, dc	*Not used*	-
	SENSOR	FLUOROMETER_CHLA
	SENSOR_MAKER	WETLABS
	SENSOR_MODEL	ECO_FLBBCD
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_ECO.CHANNEL_02		

sf, dc	*Not used*	-
	SENSOR	FLUOROMETER_CDOM
	SENSOR_MAKER	WETLABS
	SENSOR_MODEL	ECO_FLBBCD
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_ECO.CHANNEL_03		
sf, dc	*Not used*	-
	SENSOR	BACKSCATTERINGMETER_BBP700
	SENSOR_MAKER	WETLABS
	SENSOR_MODEL	ECO_FLBBCD
	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_SBEPH		
SENSOR_SBEPH.SENSOR		
	SENSOR	TRANSISTOR_PH
	SENSOR_MAKER	SBE
	SENSOR_MODEL	SEAFET
SN	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_CROVER		
SENSOR_CROVER.SENSOR		
	SENSOR	TRANSMISSOMETER_CP660
	SENSOR_MAKER	WETLABS
	SENSOR_MODEL	C_ROVER
SN	SENSOR_SERIAL_NUMBER	*SN*
SENSOR_CROVER.PATH_LENGTH		
pth	*Not used*	-
SENSOR_CROVER.CALIBRATION		
cln	*Not used*	-
SENSOR_SUNA		
SENSOR_SUNA.SENSOR		
	SENSOR	SPECTROPHOTOMETER_NITRATE
	SENSOR_MAKER	SATLANTIC
	SENSOR_MODEL	SUNA_V2
SN	SENSOR_SERIAL_NUMBER	*SN*
Model	*To convert*	*SUNA-V2*
Spectrum	*Not used*	-
SENSOR_SUNA.SUNA_BOARD		
Firmware	*Not used*	-
SENSOR_SUNA.SPECTROMETER		
spintper	*Not used*	-
SENSOR_UVP6		
SENSOR_UVP6.SENSOR		
	SENSOR	CAMERA_PARTICLES
	SENSOR_MAKER	HYDROPTIC

Model	SENSOR_MODEL	*Model*
SN	SENSOR_SERIAL_NUMBER	*SN*

nke Instrumentation
6, rue Gutenberg
ZI de Kerandré
56700 Hennebont
FRANCE
Phone: +33 (0)2 97 36 10 12

www.nke-instrumentation.com

